Lecture 22

# LLM Training and Applications

Pre-training, post-training, and using LLMs.

**EECS 189/289, Fall 2025 @ UC Berkeley**
Joseph E. Gonzalez and Narges Norouzi

# Recap – Causal Language Modeling

# Predicting the Next Word is *Knowledge*

9688650

The capital of California is **Sacramento**

San Francisco (1862)
Benicia (1853)
Vallejo (1852)

Predicting the next word allows you to:

- Answer questions
- Tell stories
- Accomplish tasks

→ **Generative AI**

**How do we *model* the next ~~word~~ token?**

# Quick Recap

9688650

- **Causal language modeling** predict next token given context

**Model:** Pr( "EECS-189" | "The best class at UC Berkeley is"])

Next Token

Context (ordered tokens)

- **Auto-regressive** (iterative) decoding:

The best class at UC Berkeley **is**
The best class at UC Berkeley is **EECS-189**
The best class at UC Berkeley is EECS-189**!**
The best class at UC Berkeley is EECS-189!**<stop>**

# Predicting the Next Token with a Transformer

# Modeling the next token

9688650

**Model:** Pr("hat" | "the cat in the"])

Next Token

Context (ordered tokens)

**Step 1: Tokenization of the Context**

# Modeling the next token

9688650

**Model:** Pr("hat" | "the cat in the"])

Next Token

Context (ordered tokens)

**Step 1: Tokenization of the Context**

| | | | | |
|---|---|---|---|---|
| | the | cat | in | the |
| **Token Id:** | 142 | 307 | 153 | 142 |

# Embedding Table Lookup

9688650

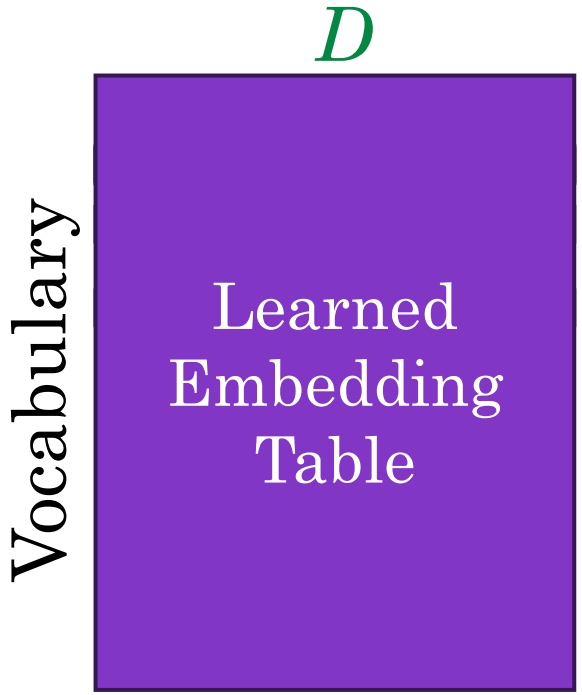
Lookup **embedding** is **learned embedding table**.

the → 142

cat → 307

in → 153

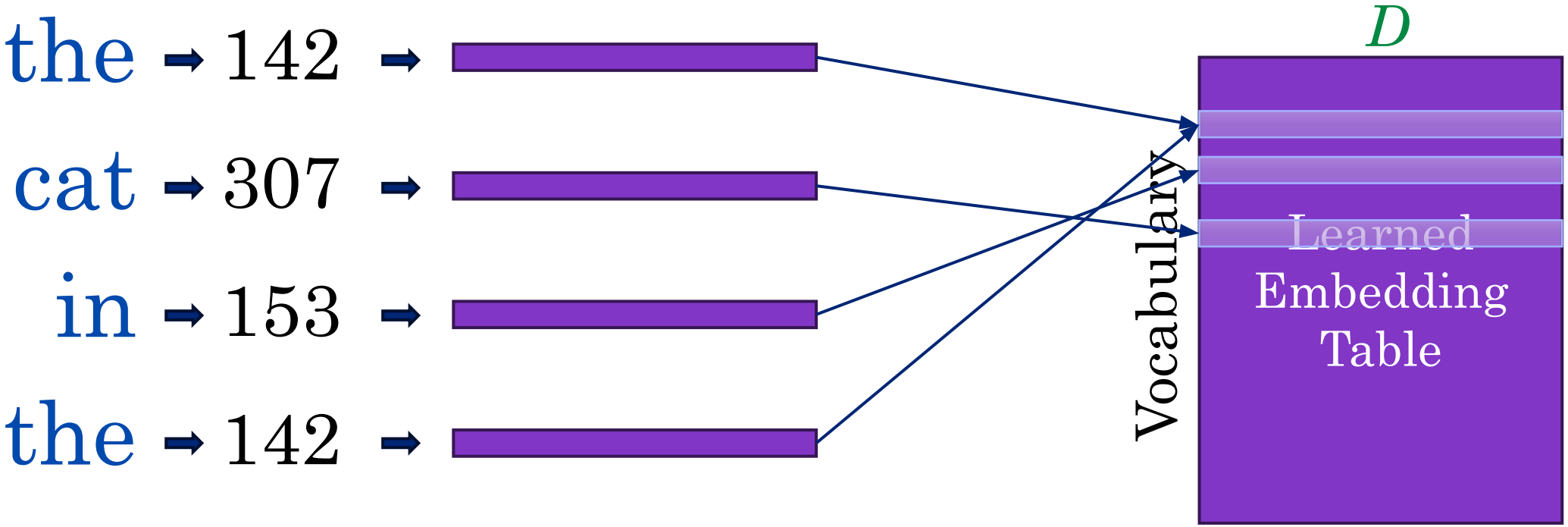
the → 142



# Embedding Table Lookup

9688650

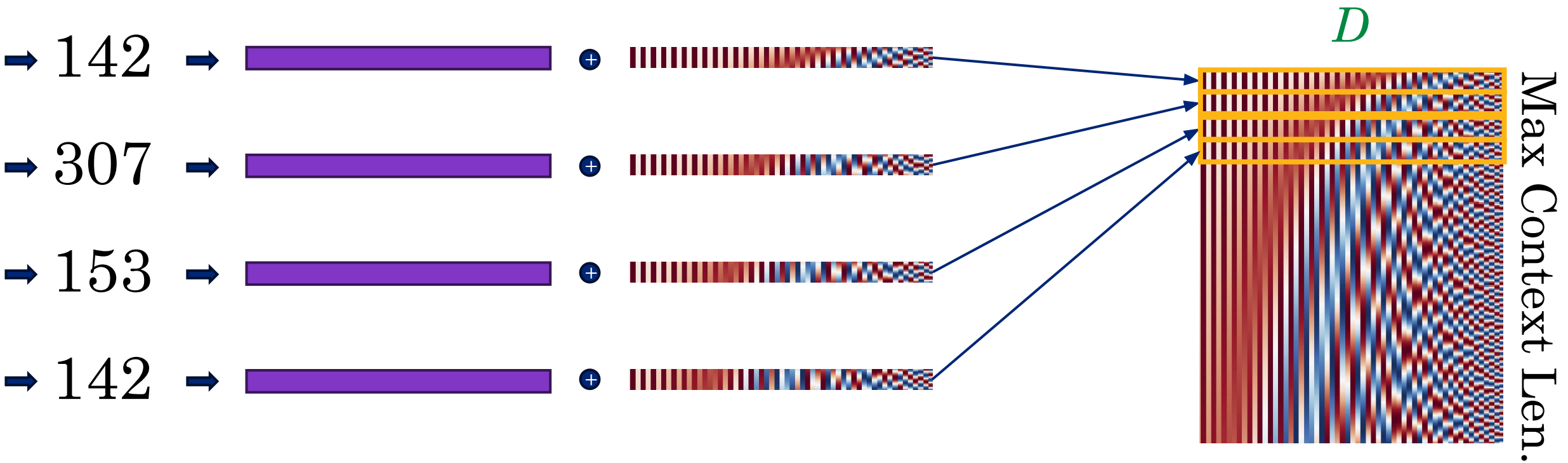
Lookup **embedding** is **learned embedding table**.



# Positional Encoding (optional)

9688650

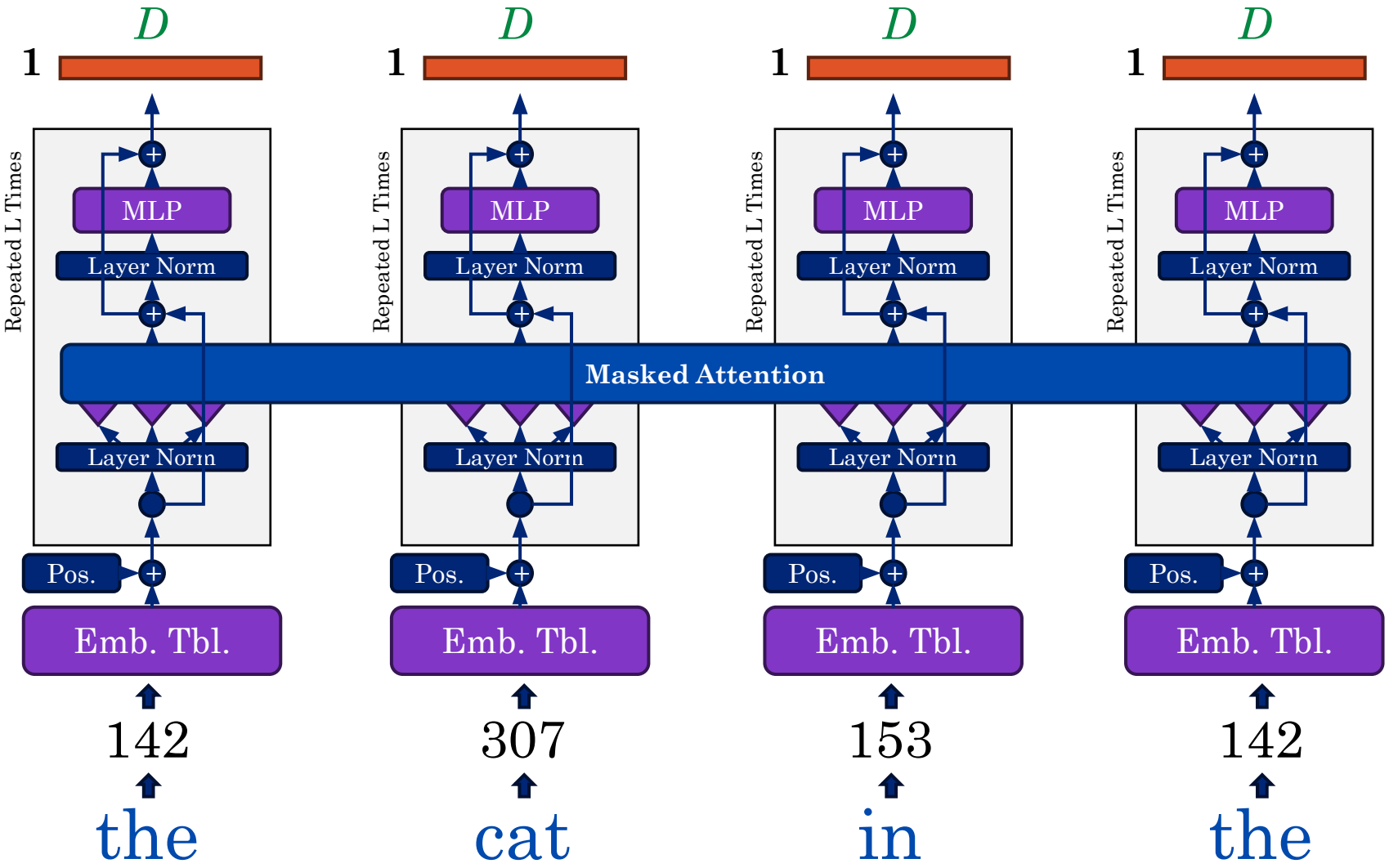
Add fixed positional encoding to embeddings based on position



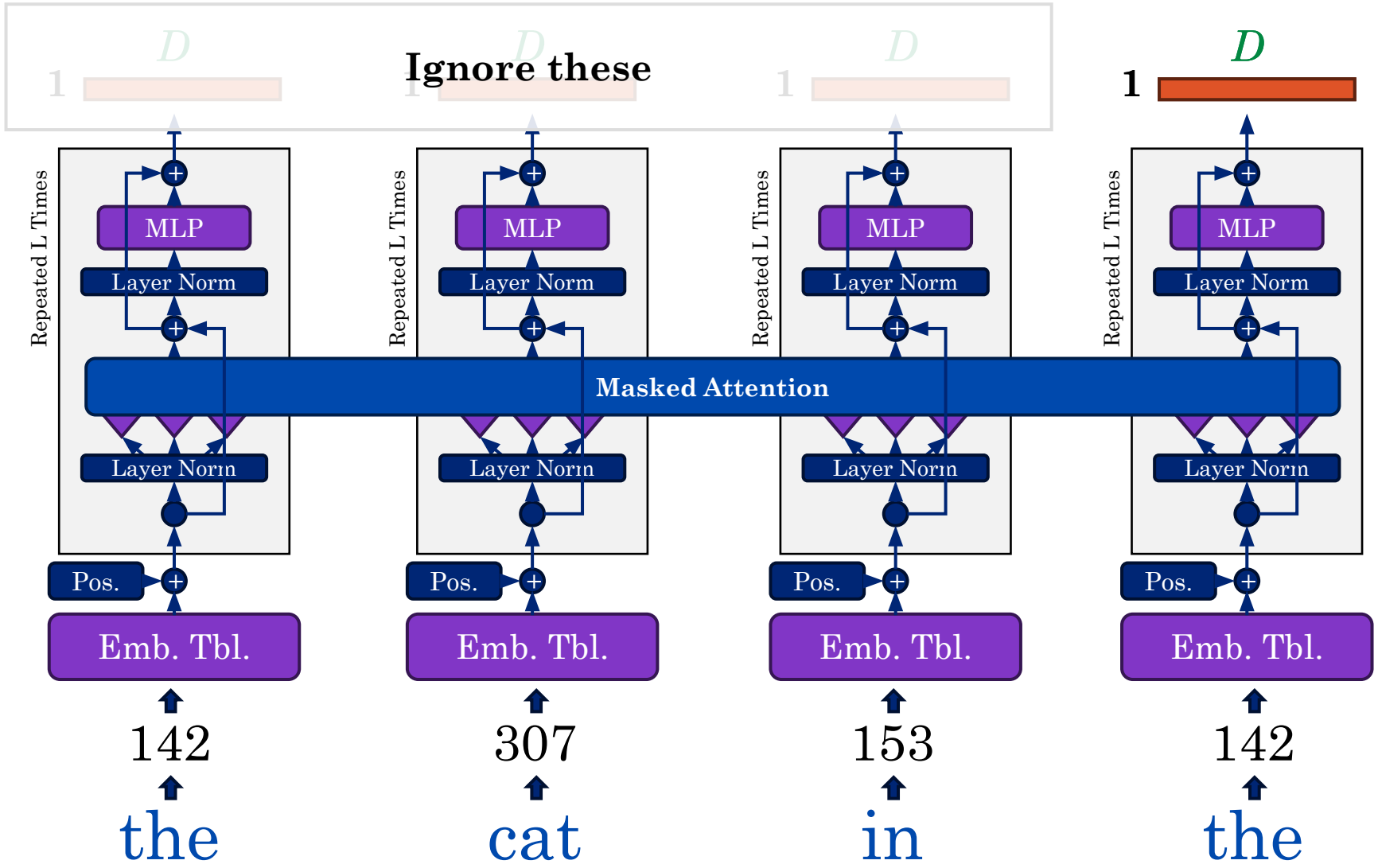

**Pass into transformer architecture**

# Predicting the Next Token (Transformer)

9688650



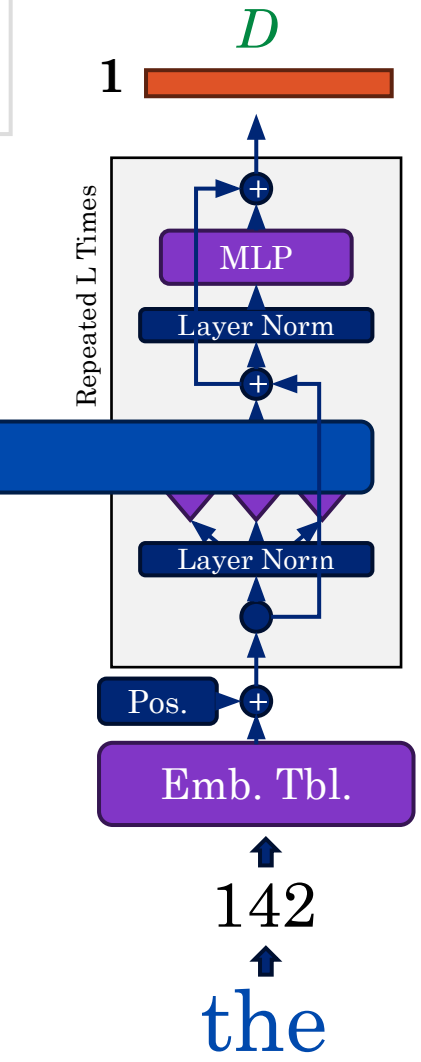
# Predicting the Next Token (Transformer)

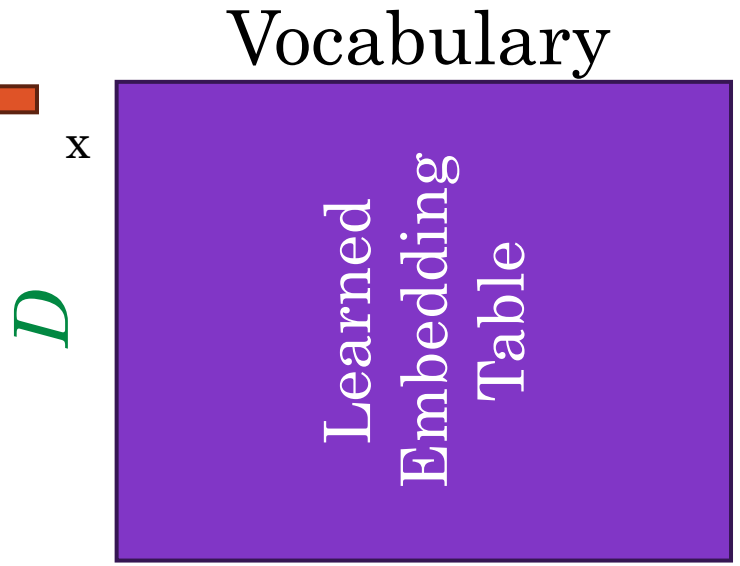


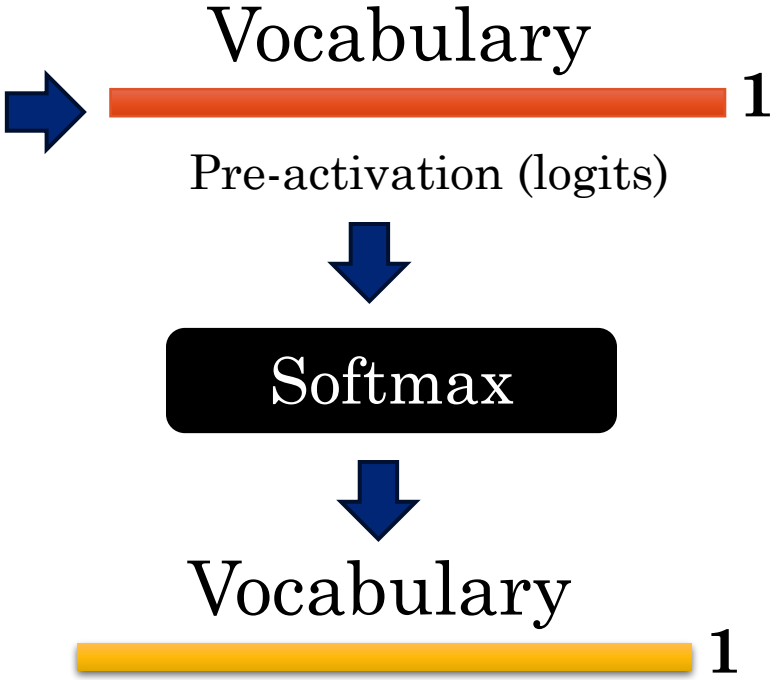

9688650

# Predicting the Next Token (Transformer)

9688650




**Probability** that each token is the **next token**

# Training the Model

# What is Chat GPT?

9688650

**Chat**: natural language system

✅ **G**: **Generatively** – Designed to model the creation of text

**P**: **Pretrained** – Trained on lots of naturally occurring data

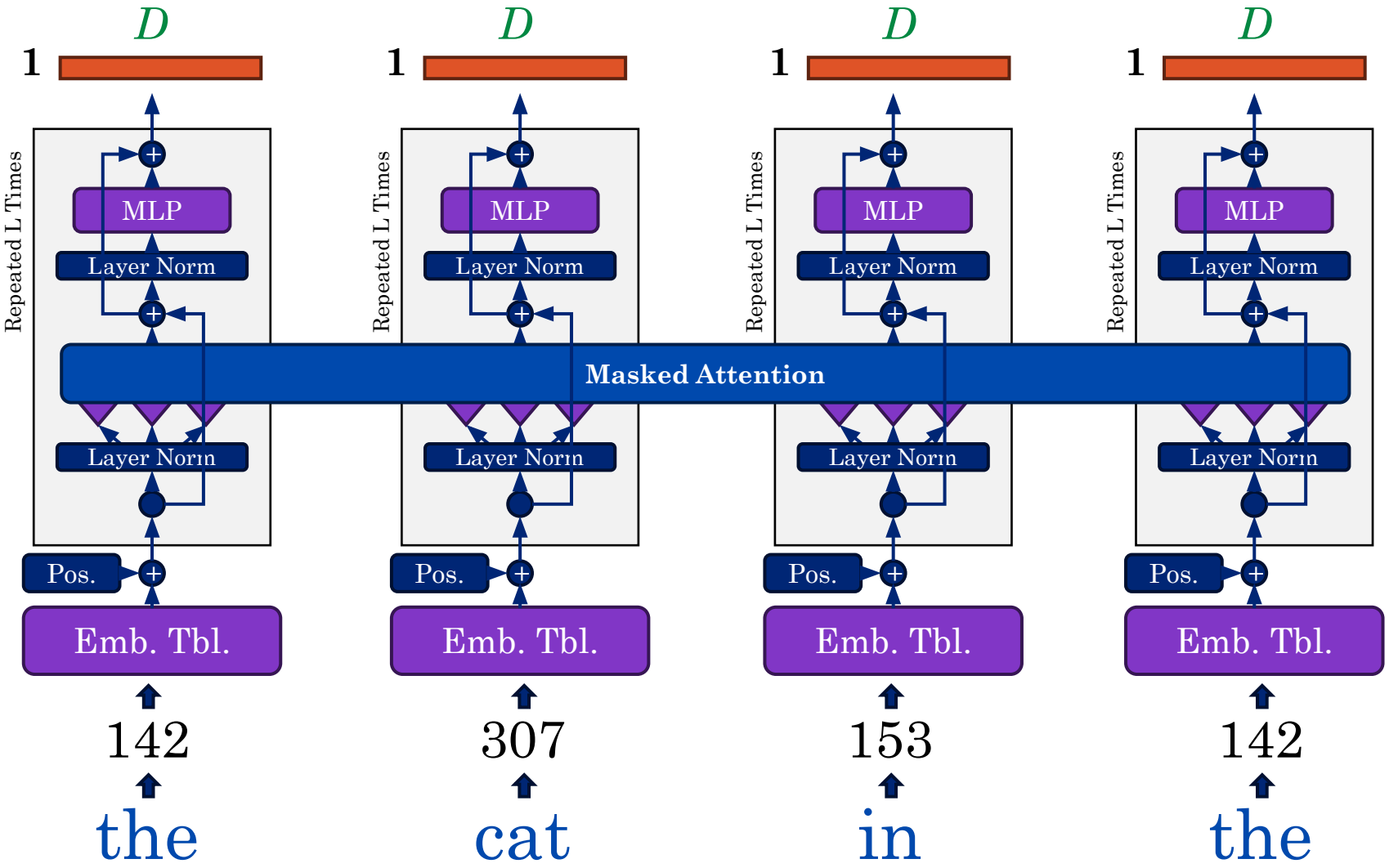
✅ **T**: **Transformer** – A kind of neural network architecture

Chat GPT is just one example of a

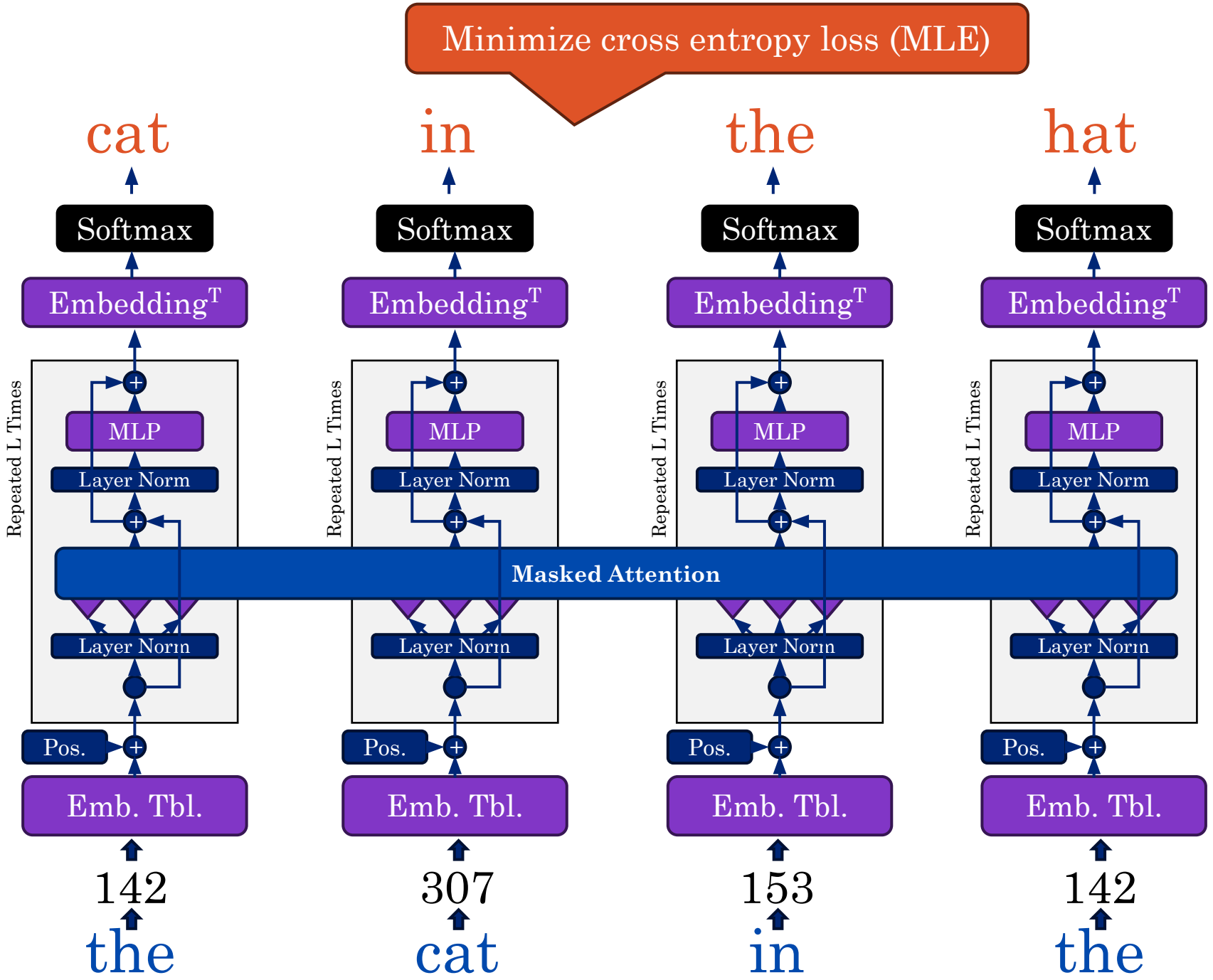
✅ **Large Language Model (LLM)**

# Training the Model

9688650

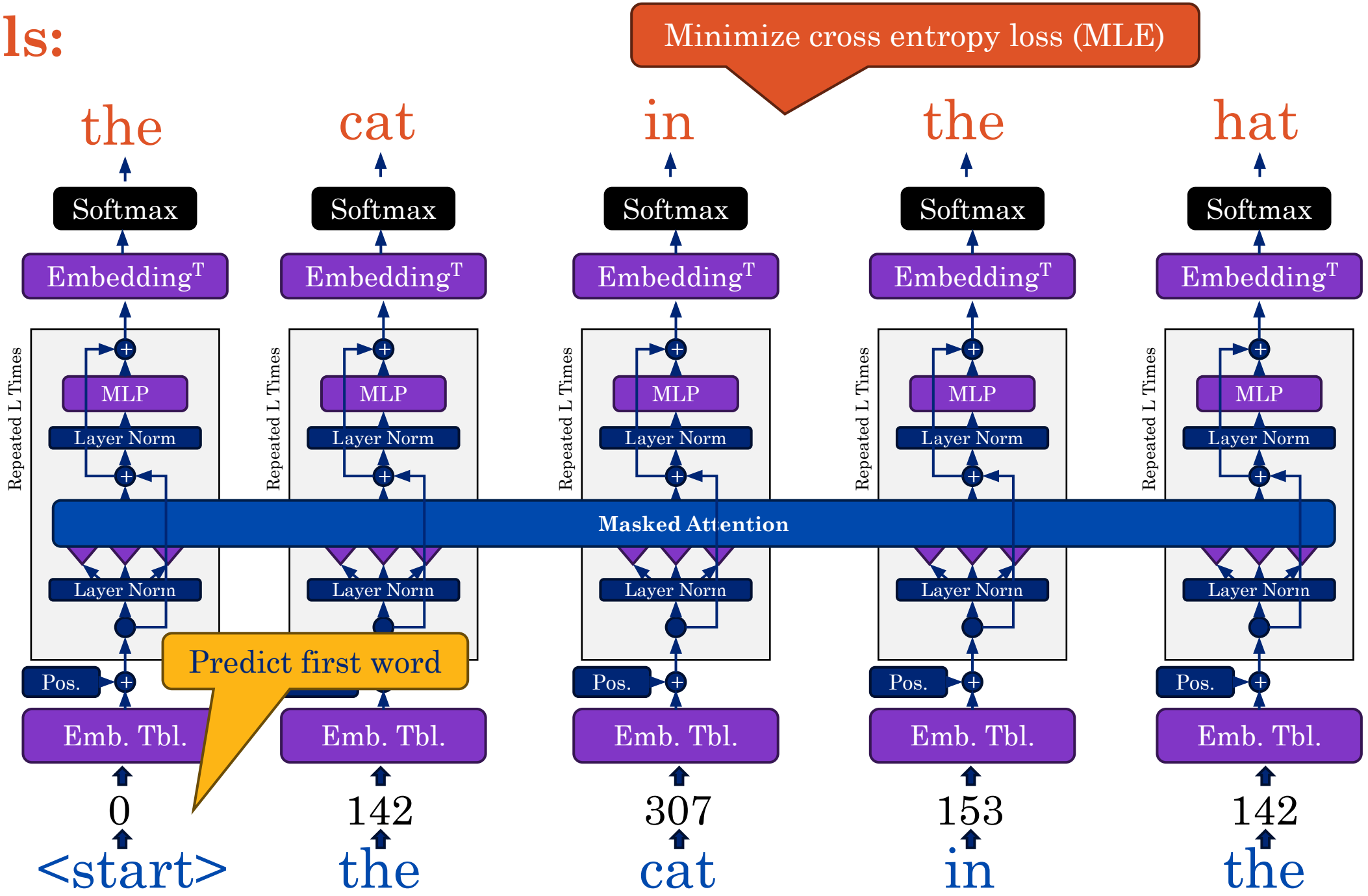


# Training the Model




9688650

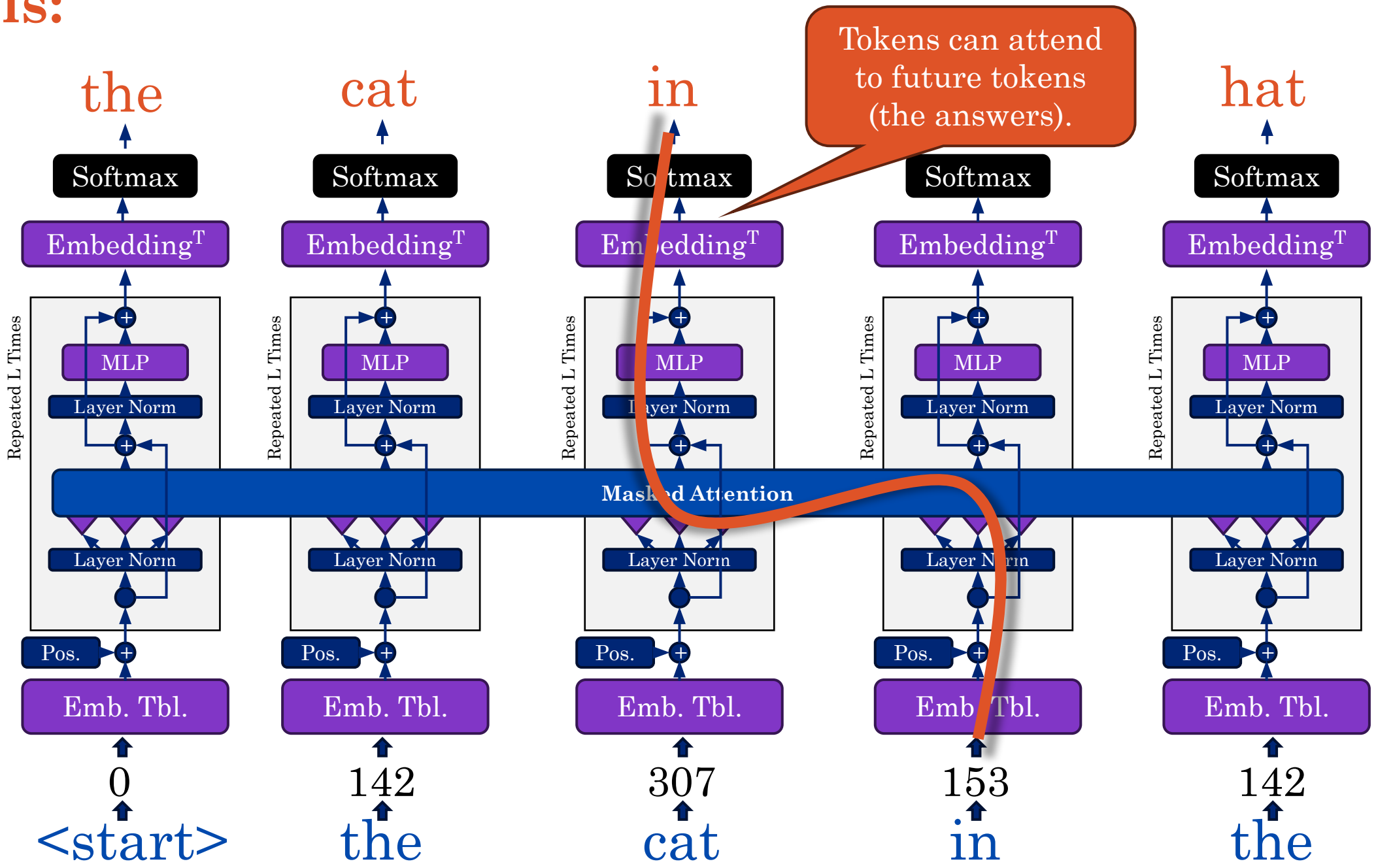
# Labels:




9688650

# For a sentence with N tokens how many MLE prediction tasks are there?

# Labels:



# The Masked Attention

$$\text{Attn}[\mathbf{K},\mathbf{Q},\mathbf{V}]_n = (\boldsymbol{\alpha}\,\mathbf{V})_\mathbf{n} = \sum_{i=1}^{n} \alpha_{ni}\mathbf{V}_i = \sum_{i=1}^{n} \left( \text{SoftMax}_{\text{row}}\left[\frac{\mathbf{Q}\,\mathbf{K}^\top}{\sqrt{D_k}}\right]\right)_{ni} \mathbf{V}_i$$

**Symbols:**

$\mathbf{V} = \mathbf{X}\,\mathbf{W}^{(v)}$

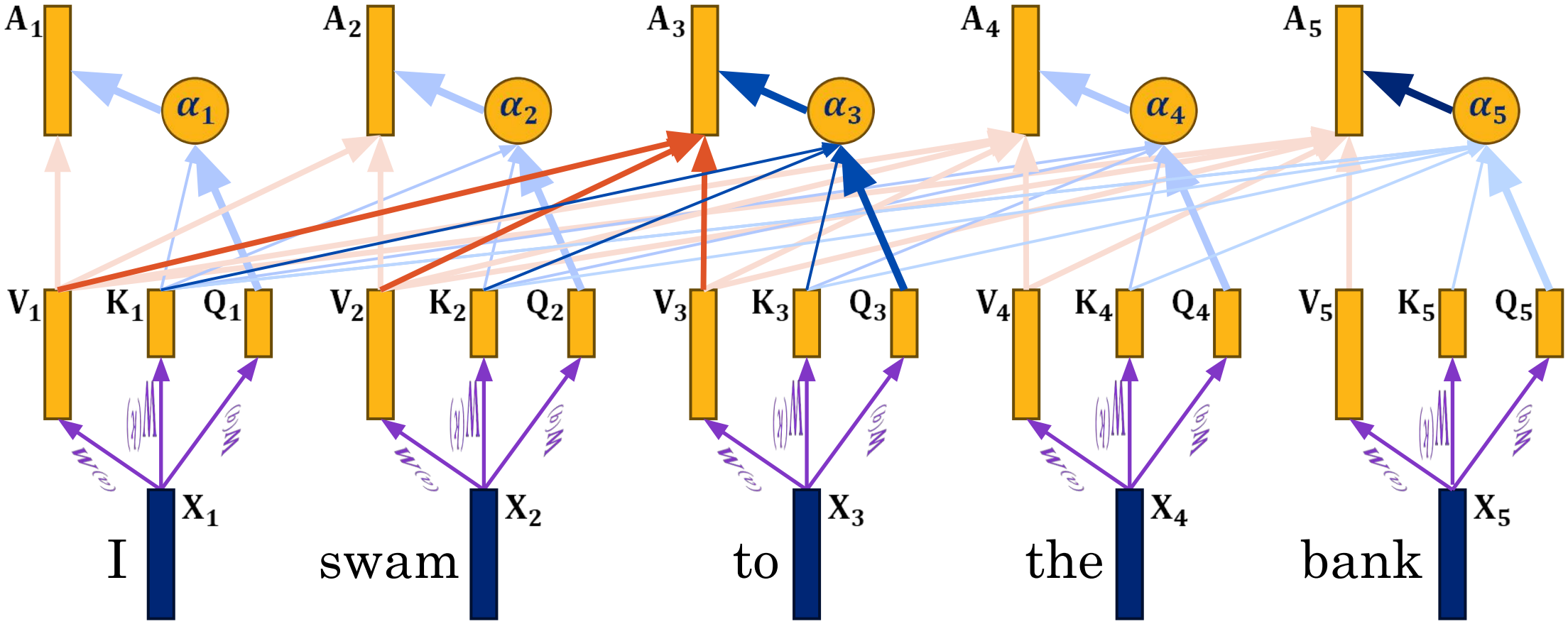
$\mathbf{K} = \mathbf{X}\,\mathbf{W}^{(k)}$

$\mathbf{Q} = \mathbf{X}\,\mathbf{W}^{(q)}$

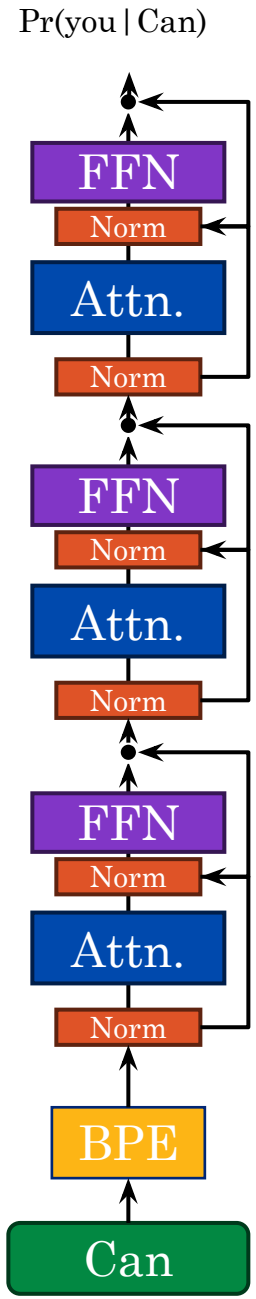
$\boldsymbol{\alpha} = \text{SoftMax}_{\text{row}}\left[\frac{\mathbf{Q}\,\mathbf{K}^\top}{\sqrt{D_k}}\right]$

$\boldsymbol{\alpha} \in \mathbb{R}^{N\times N}$

9688650

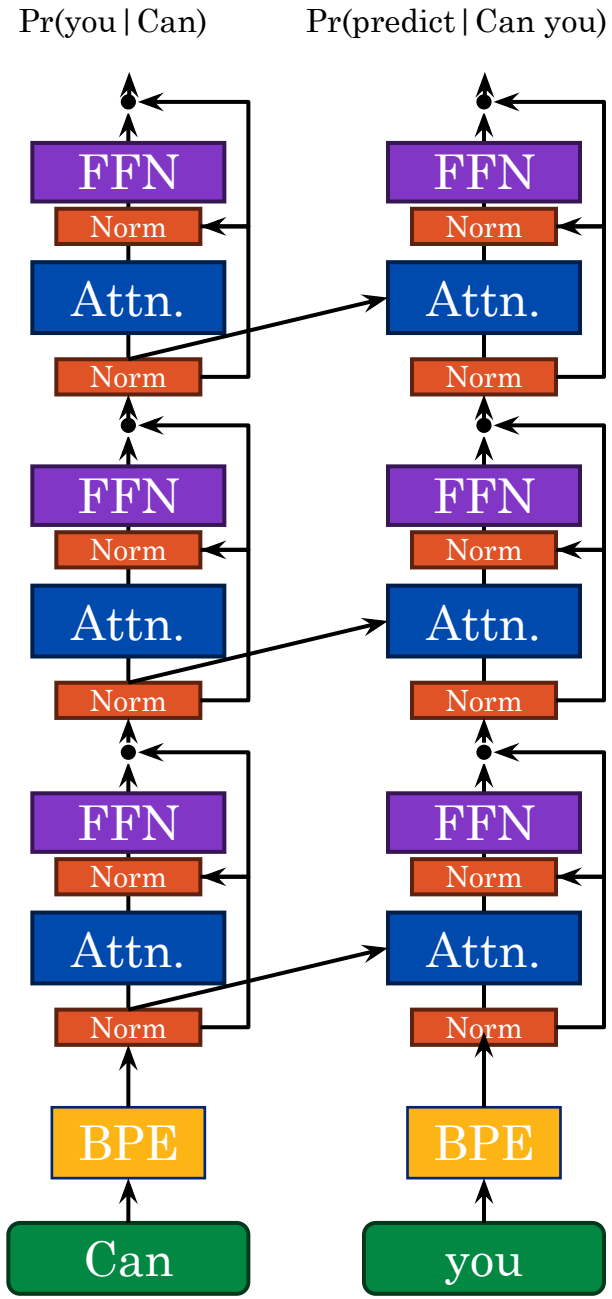


# Unrolling the Model

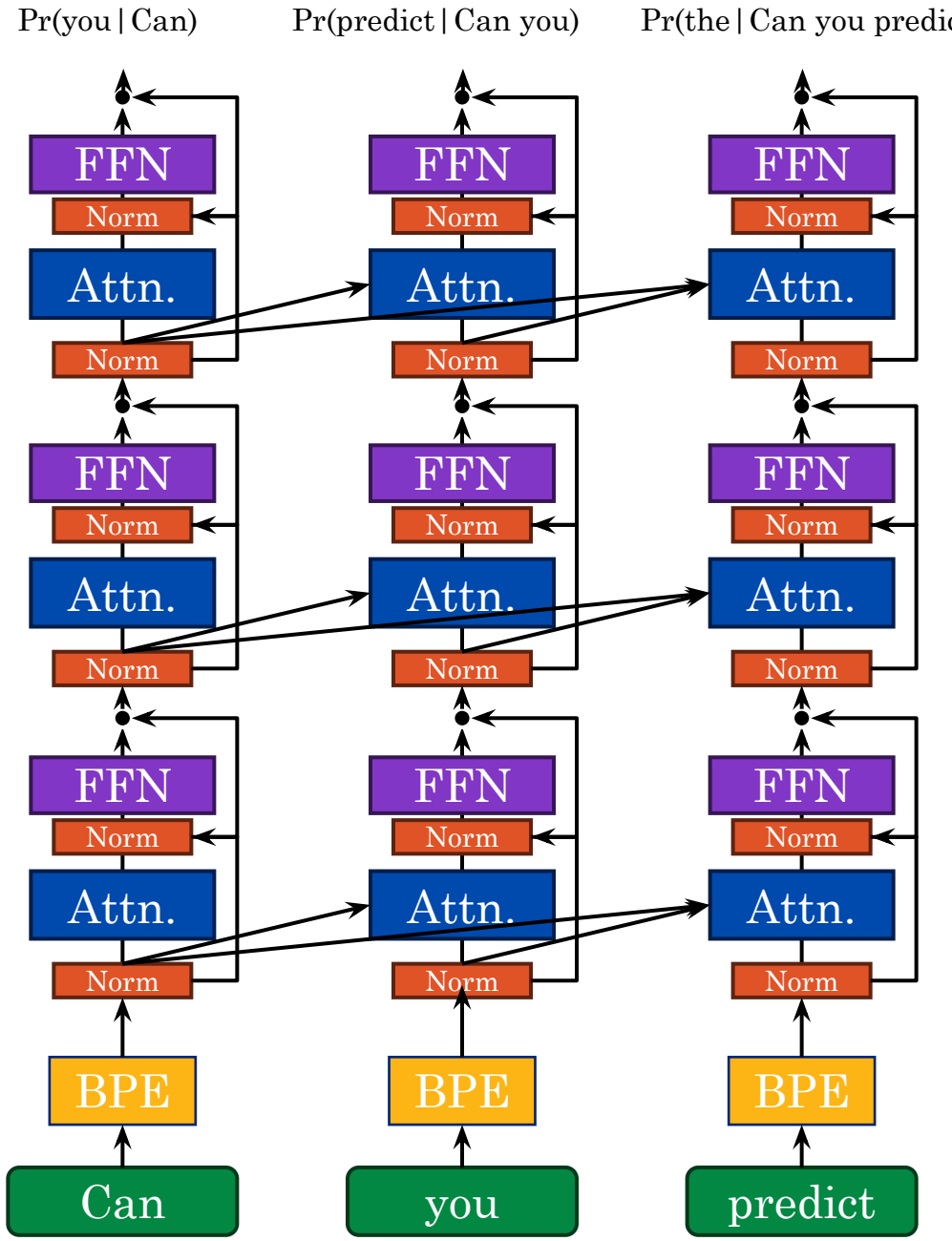



9688650

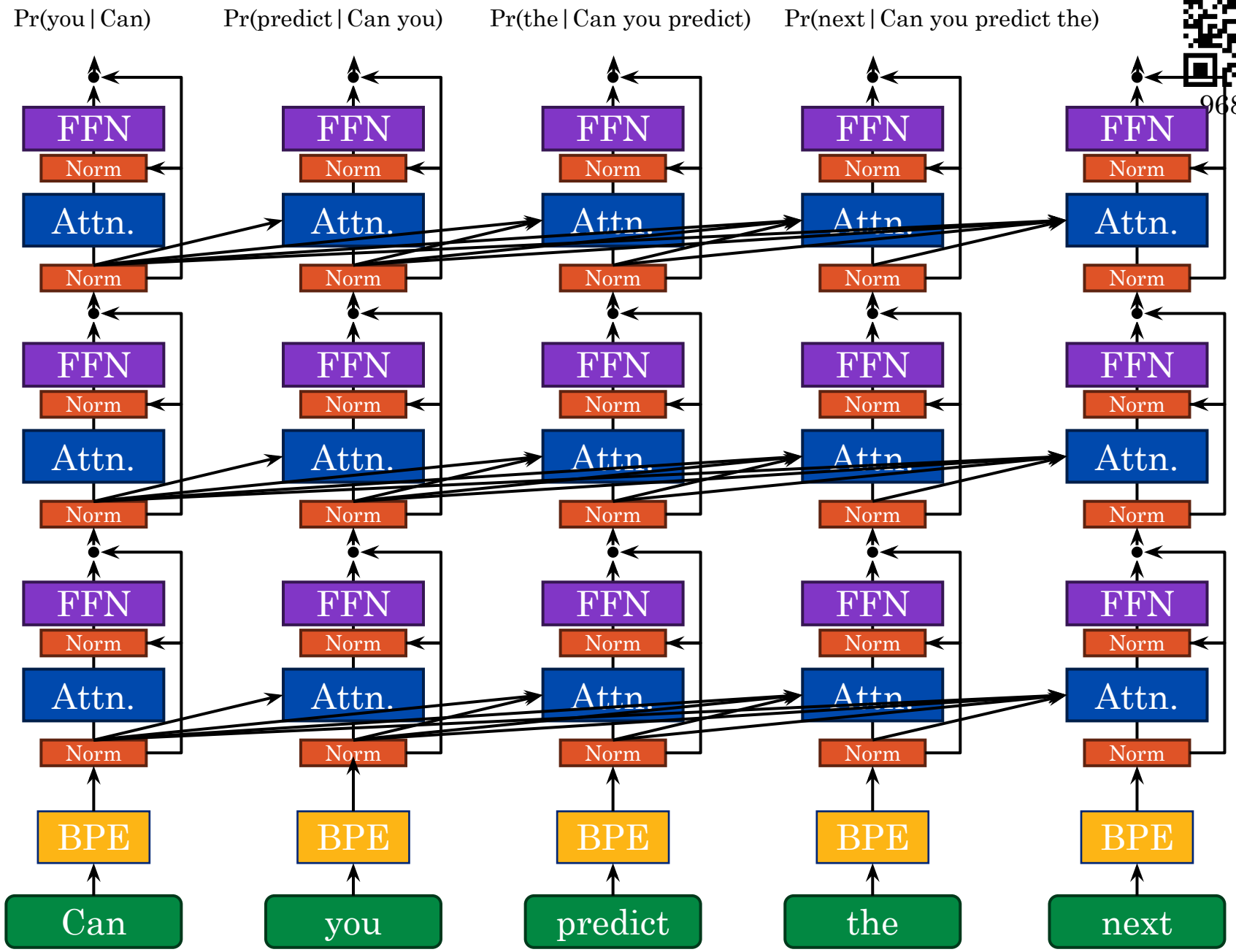
# Unrolling the Model



# Unrolling the Model



# Unrolling the Model

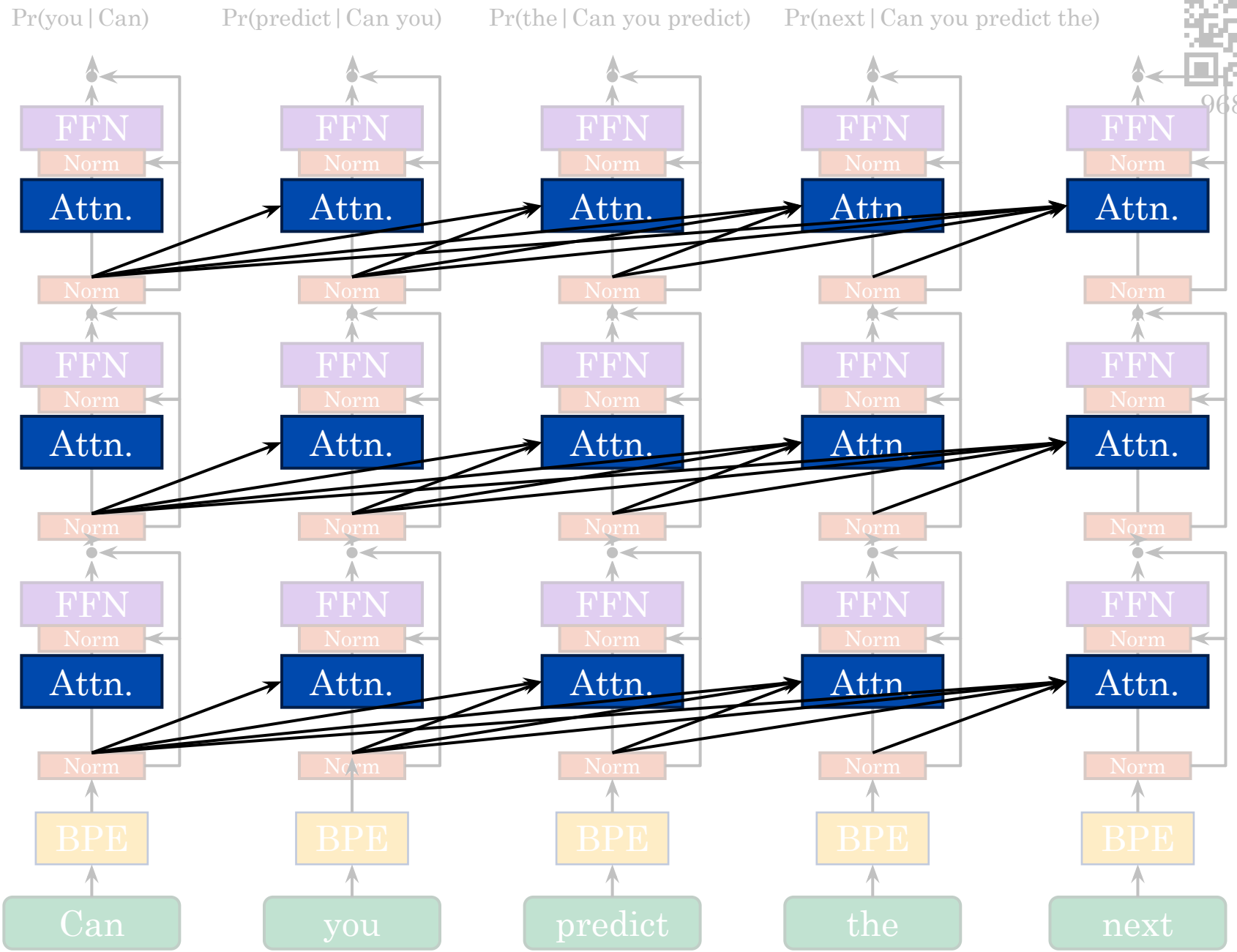


# Masked Attention

Auto-regressive

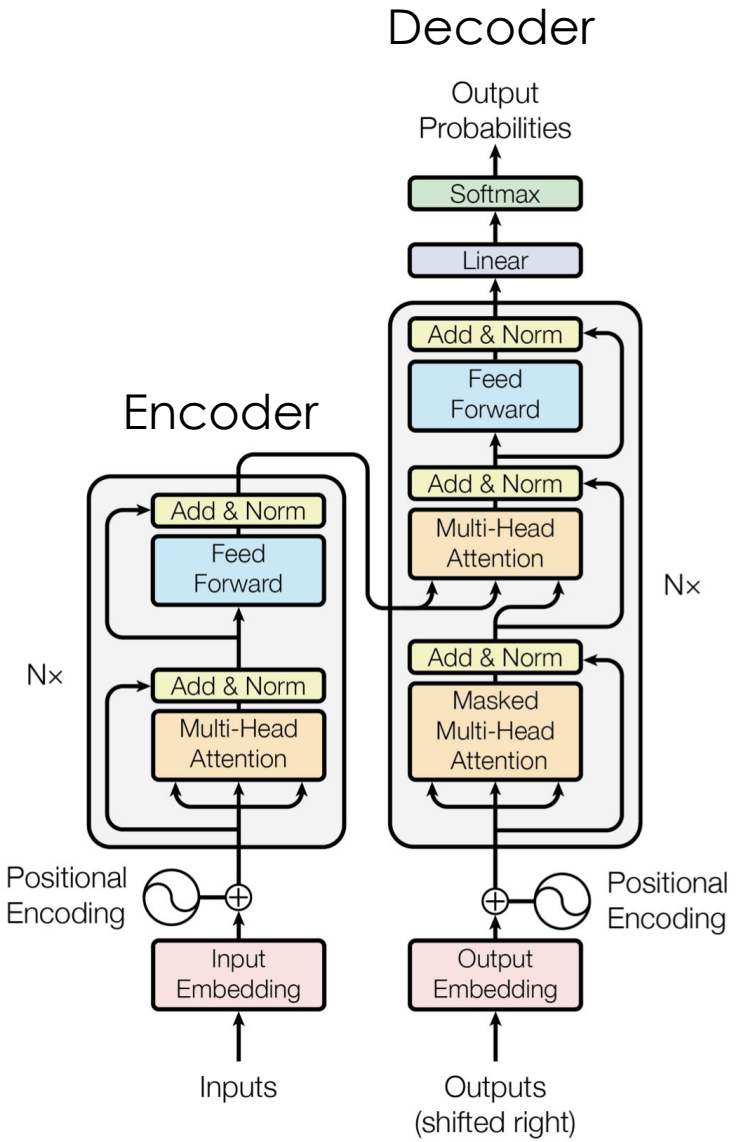
Attend only to previous tokens

**Decoder** only Transformer



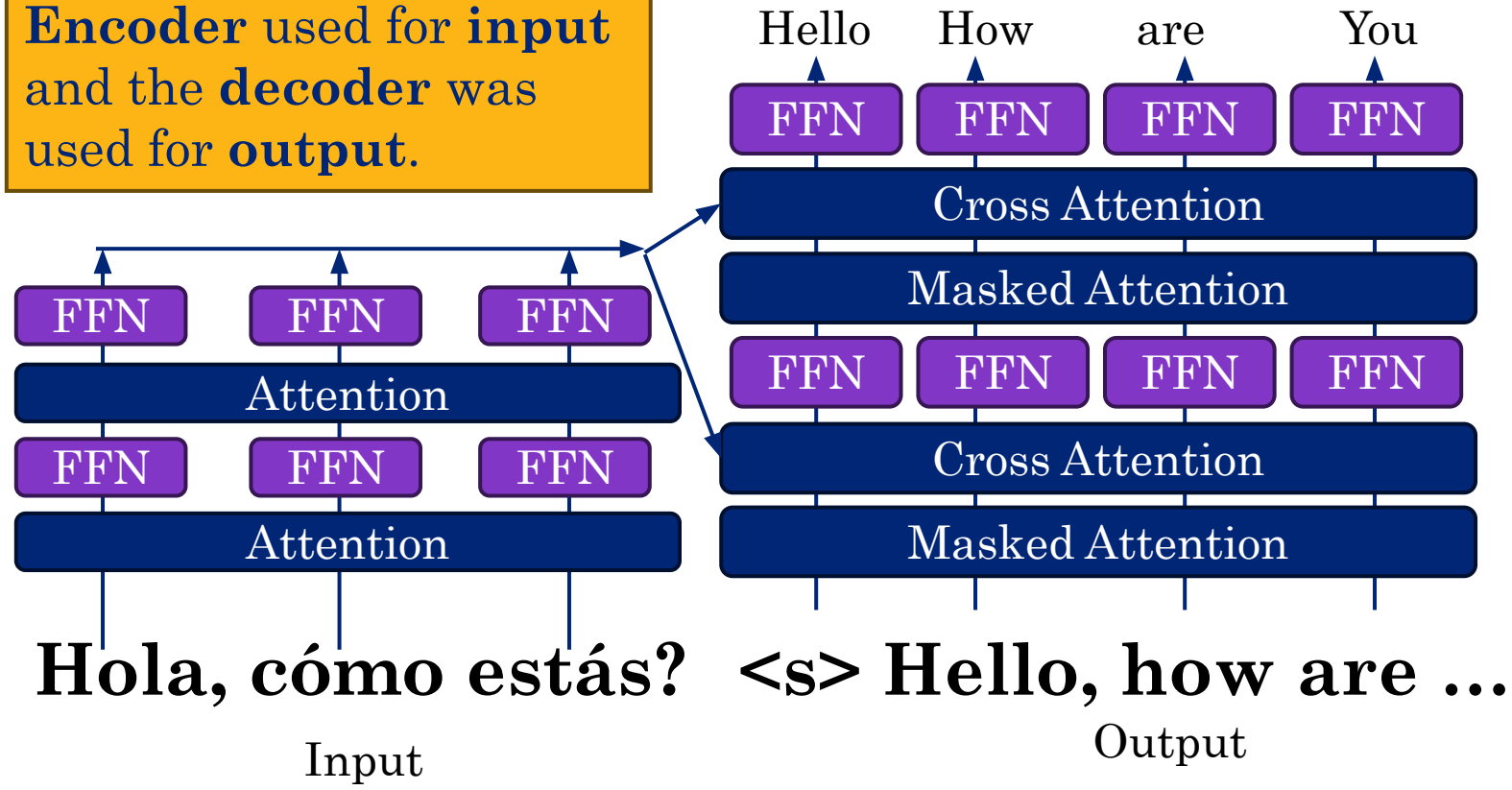
# The Encoder-Decoder Transformer

9688650




The original transformer paper used both an **encoder** and a **decoder**. Why?

**Encoder** used for **input** and the **decoder** was used for **output**.



# Visual Language Models

9688650

Use a visual encoder to convert image patches into token embeddings that are fed to the LLM as tokens.

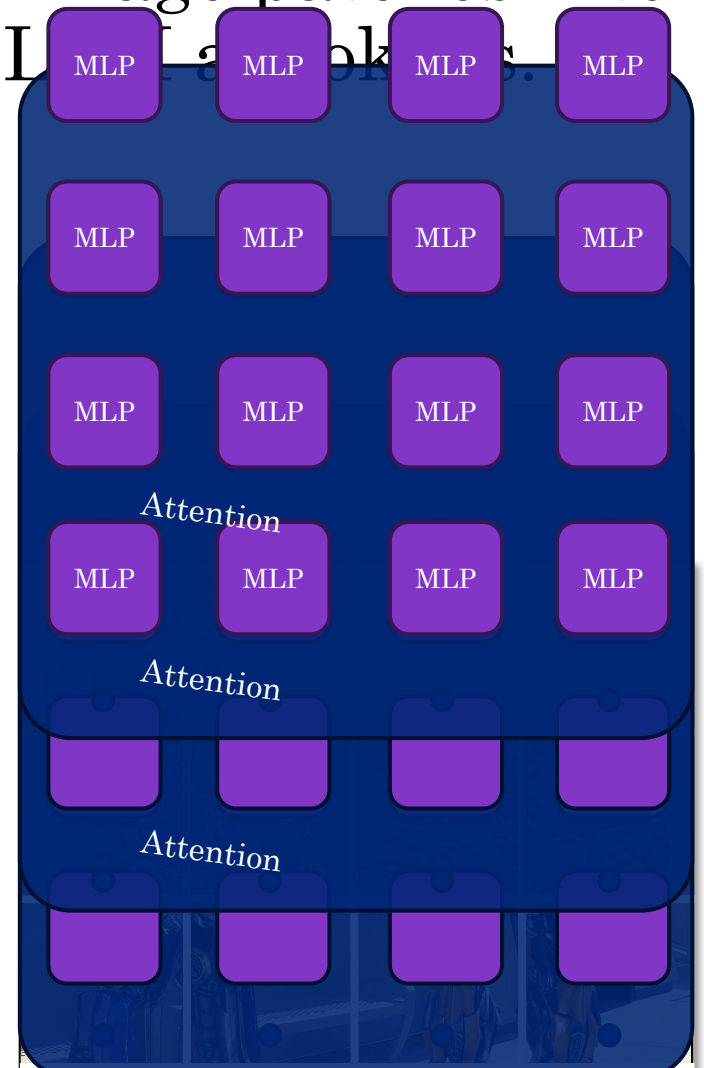
*Describe this image:*


# Visual Language Models

9688650

Use a visual encoder to convert image patches into token embeddings that are fed to the L[illegible]

*Describe this image:*

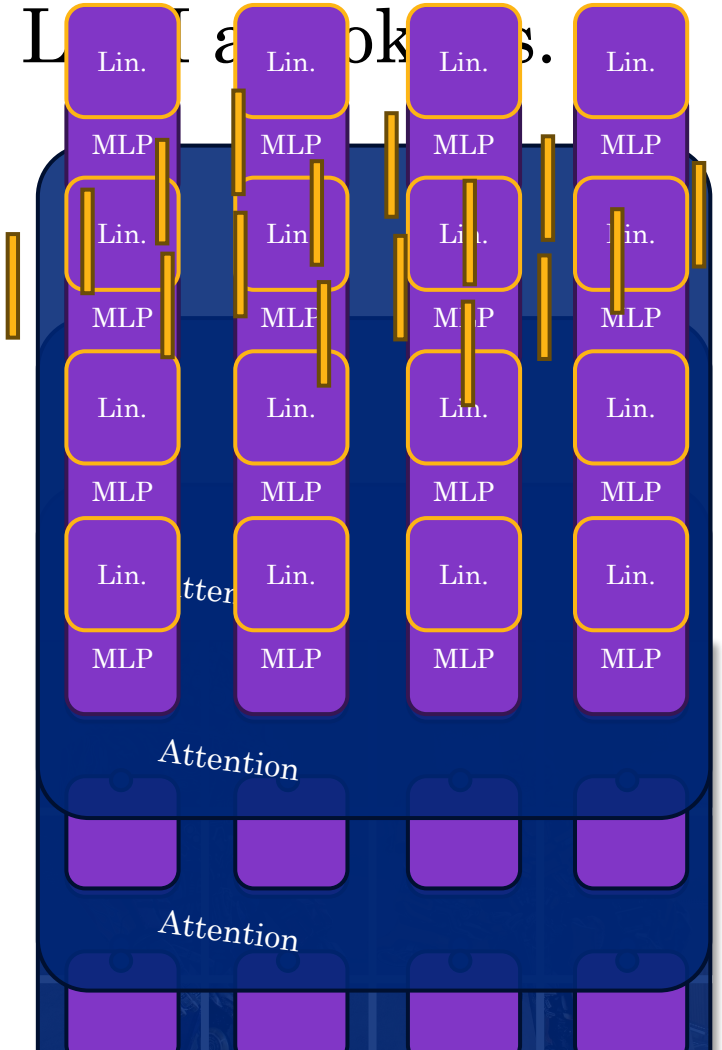


# Visual Language Models

9688650

Use a visual encoder to convert image patches into token embeddings that are fed to the L[illegible] a[illegible]ok[illegible]s.

*Describe this image:*

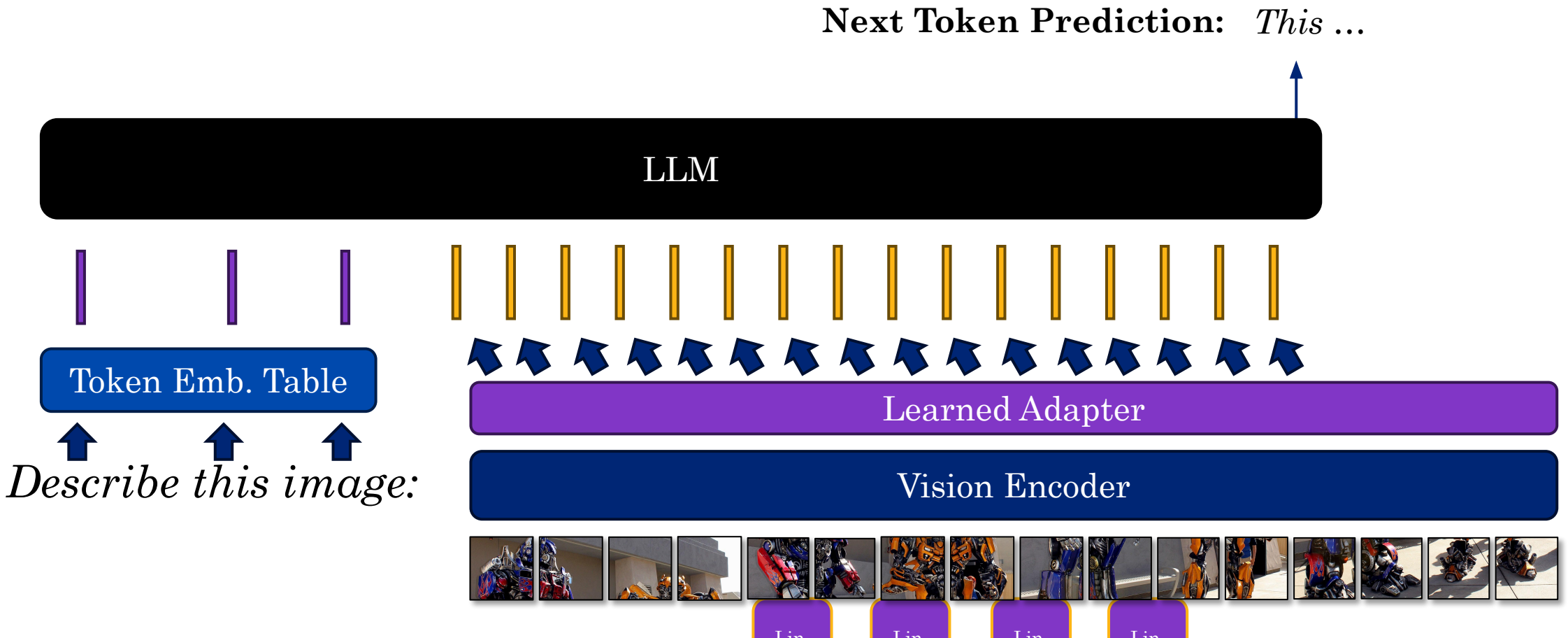



Learn “adapter” to transform image embeddings to token embeddings.

# Visual Language Models

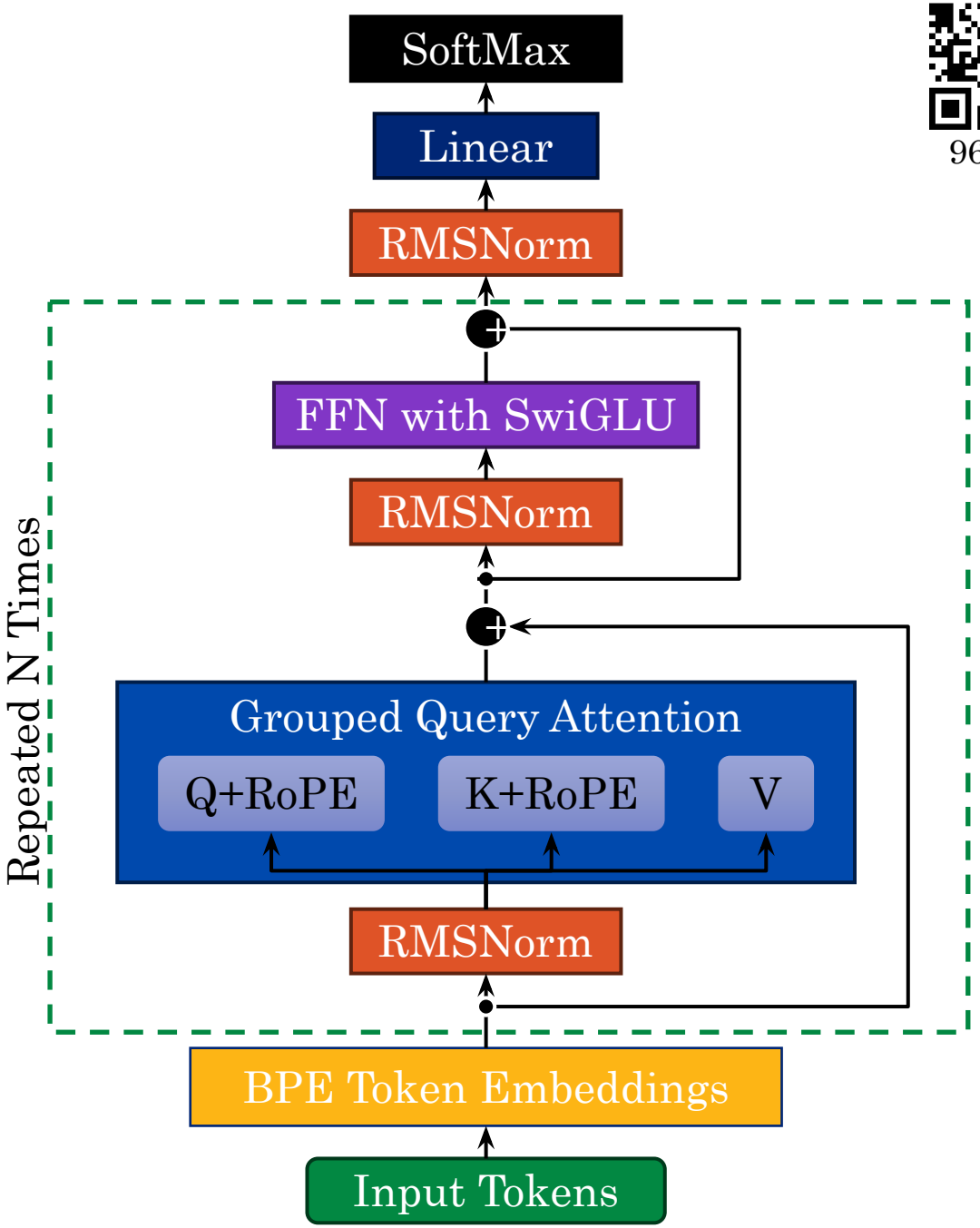
9688650

Use a visual encoder to convert image patches into token embeddings that are fed to the LLM as tokens.



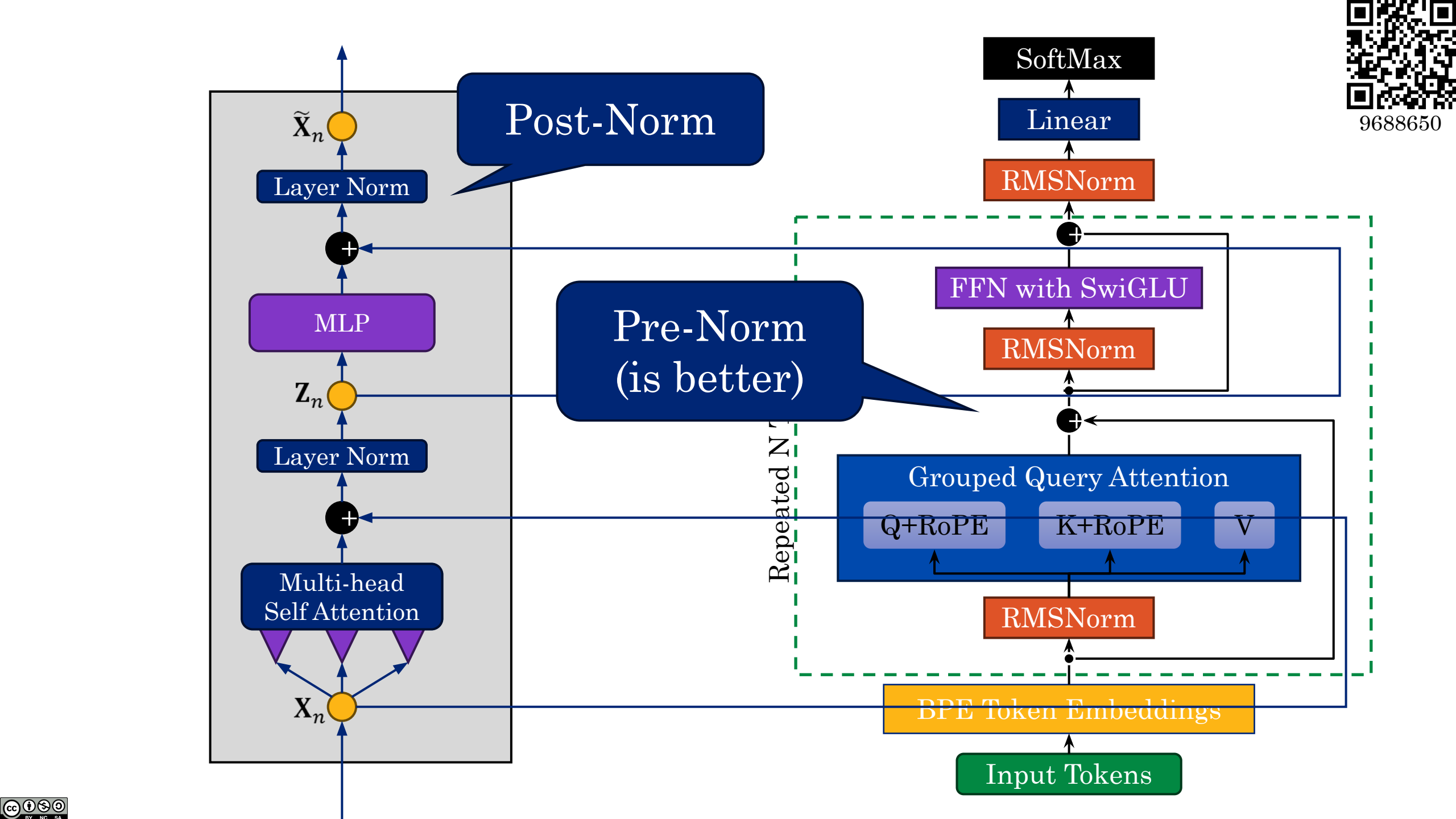
# Llama-3 Architecture

- **RMSNorm** (Layer Norm.) –improve training stability
- **FFN with SwiGLU** – Feed forward network
- **Residual Connections** – improve training stability
- **Repeated N Times** –increase model size




9688650

See Actual Code (its just one python script!)


Post-Norm
$\widetilde{\mathbf{X}}_n$
Layer Norm
MLP
$\mathbf{Z}_n$
Layer Norm
Multi-head Self Attention
$\mathbf{X}_n$
Pre-Norm (is better)
SoftMax
Linear
RMSNorm
FFN with SwiGLU
RMSNorm
Grouped Query Attention
Q+RoPE
K+RoPE
V
RMSNorm
BPE Token Embeddings
Input Tokens
Repeated N

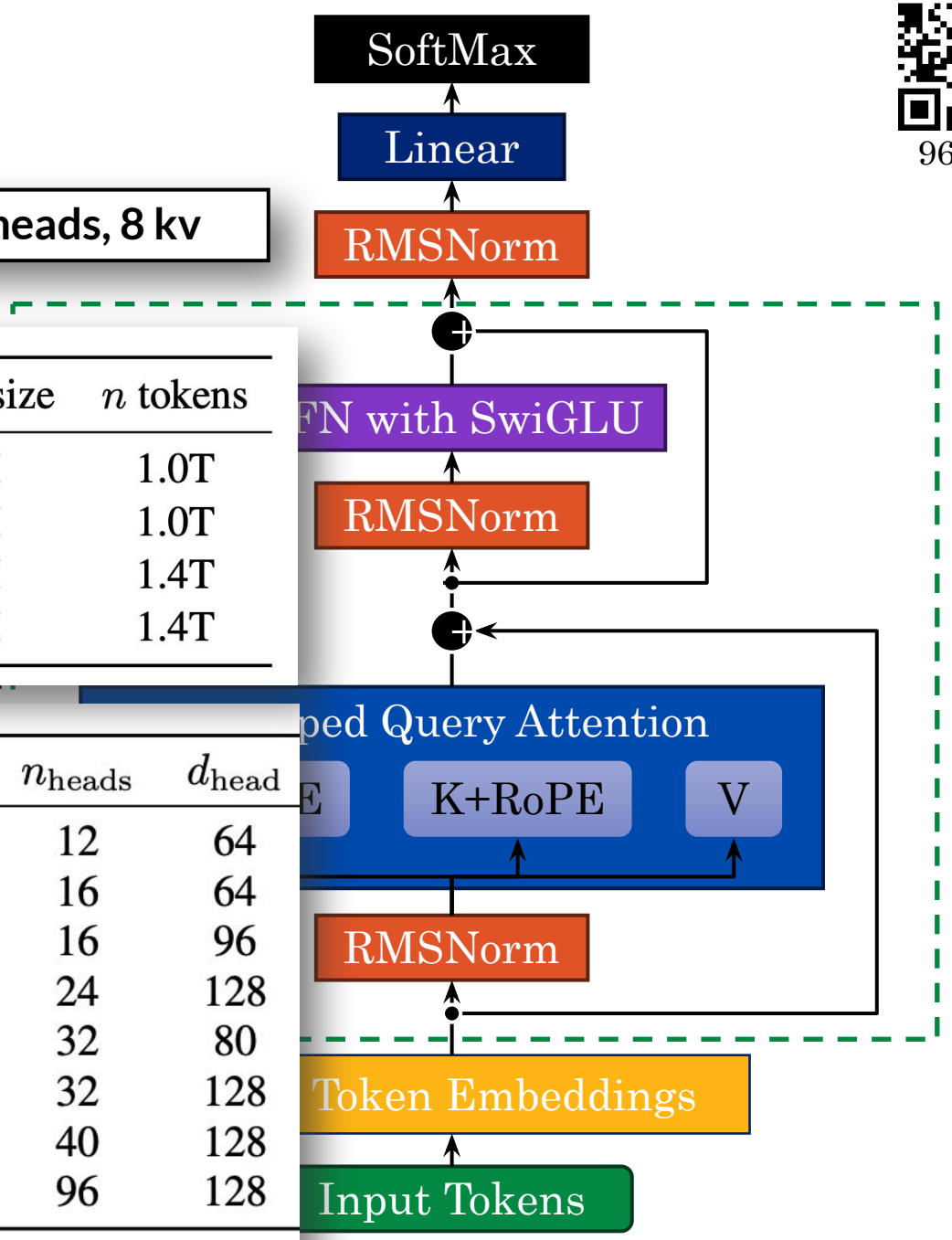
# Going Big!

**Llama-3 70b Instruct: 8192 hidden size, 80 layers, 64 query heads, 8 kv heads.**

Llama 2

| params | dimension | $n$ heads | $n$ layers | learning rate | batch size | $n$ tokens |
|---|---|---|---|---|---|---|
| 6.7B | 4096 | 32 | 32 | $3.0e^{-4}$ | 4M | 1.0T |
| 13.0B | 5120 | 40 | 40 | $3.0e^{-4}$ | 4M | 1.0T |
| 32.5B | 6656 | 52 | 60 | $1.5e^{-4}$ | 4M | 1.4T |
| 65.2B | 8192 | 64 | 80 | $1.5e^{-4}$ | 4M | 1.4T |

| Model Name | $n_{\text{params}}$ | $n_{\text{layers}}$ | $d_{\text{model}}$ | $n_{\text{heads}}$ | $d_{\text{head}}$ |
|---|---|---|---|---|---|
| GPT-3 Small | 125M | 12 | 768 | 12 | 64 |
| GPT-3 Medium | 350M | 24 | 1024 | 16 | 64 |
| GPT-3 Large | 760M | 24 | 1536 | 16 | 96 |
| GPT-3 XL | 1.3B | 24 | 2048 | 24 | 128 |
| GPT-3 2.7B | 2.7B | 32 | 2560 | 32 | 80 |
| GPT-3 6.7B | 6.7B | 32 | 4096 | 32 | 128 |
| GPT-3 13B | 13.0B | 40 | 5140 | 40 | 128 |
| GPT-3 175B or "GPT-3" | 175.0B | 96 | 12288 | 96 | 128 |




9688650

# Pre-training on *Everything**

9688650

Train the model on a **large collection** of data to learn **generalizable patterns**

| Dataset | Quantity (tokens) | Weight in training mix | Epochs elapsed when training for 300B tokens |
|---|---|---|---|
| Common Crawl (filtered) | 410 billion | 60% | 0.44 |
| WebText2 | 19 billion | 22% | 2.9 |
| Books1 | 12 billion | 8% | 1.9 |
| Books2 | 55 billion | 8% | 0.43 |
| Wikipedia | 3 billion | 3% | 3.4 |

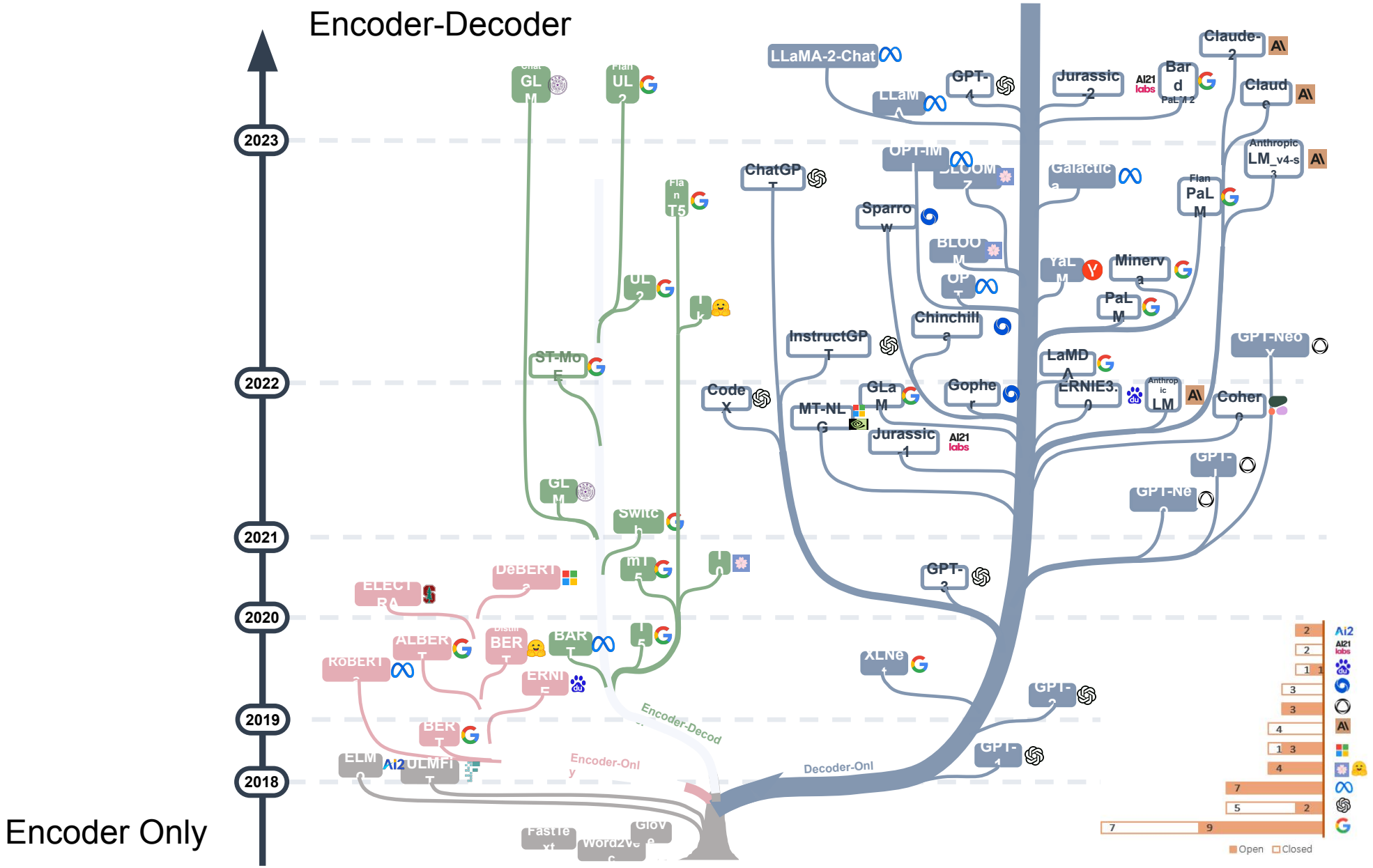
Reddit Links

OpenAI GPT3 Data Mix

Llama-3 "open-source" models trained on **15.6T tokens from an unknown data mix.**

- 405-billion parameter model trained on **16K H100s ($25K each)**
- **39.3M GPU hours**

*Everything that is legal to use for training hopefully…




https://github.com/Mooler0410/LLMsPracticalGuide/tree/main

# Demo

## Decoder Transformer Notebook

9688650

9688650

# Now you know

## What is Chat GPT

There is still one more thing

**Chat**: natural language system

- ✅ **Generatively** – Designed to model the creation of text
- ✅ **Pretrained** – Trained on lots of naturally occurring data
- ✅ **Transformer** – A kind of neural network architecture

GPT alone can't chat!

# Post-training

Tuning language models to be useful.

# Need to Teach Model to **Follow Instructions (and be helpful!)**

9688650

- Generative pre-training **captures knowledge**
  - To finish the statement "Four score and seven years ago …" you need to learn to memorize the text
- Resulting model predicts the rest of a statement.
  - "What is attorney client privilege?" the model might generate "Provide a concise answer using an example from class."
- Use **Supervised Fine-Tuning** and **RLHF** to adjust "behavior" of model to **follow instructions** and **chat like a human.**

# Supervised Fine-tuning (SFT)

9688650

Running **additional training iters.** with a specific task

The task differs from the original pre-training task

- **New objective**: translate sentence, follow instruction
- **New training data** from new source domain

Example: Chat conversations.

**Smaller learning rate** (as you get older you learn slower?)

- Avoid "catastrophic forgetting" – additional fine-tuning causes loss in pre-training knowledge and other post-training (e.g., instruction following) behaviors.

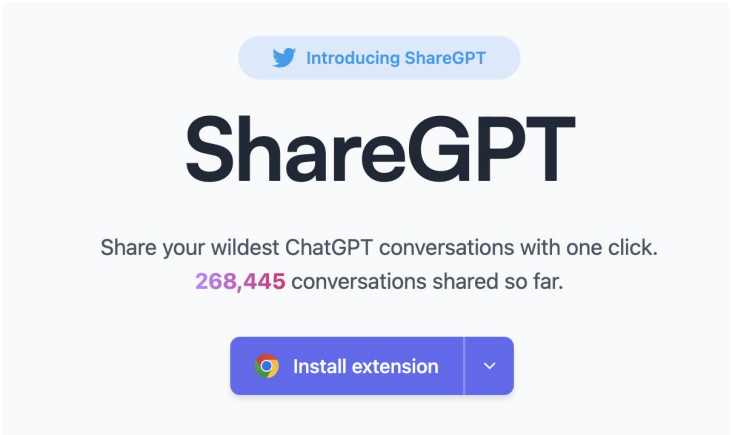
Vicuña

# Open-source Instruction Fine-Tuned LLMs and LLM-as-a-judge

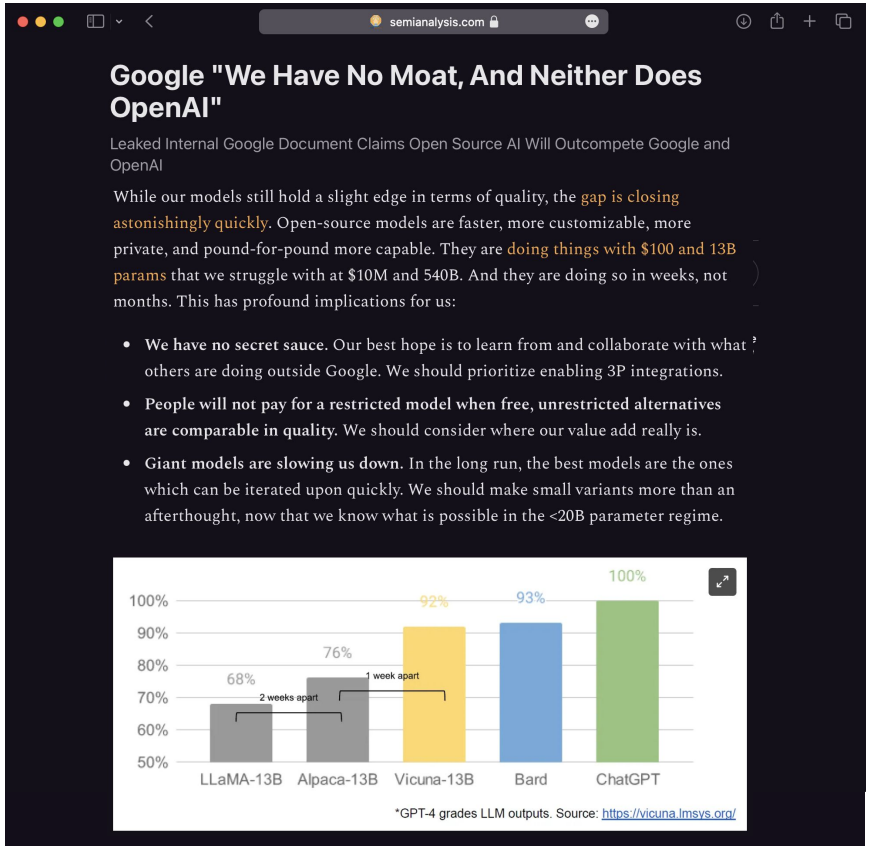
9688650

First open-source model that was **“comparable” to ChatGPT**

Fine-tuned **LlaMA-13B** on **ShareGPT Data**




**Tiny**
**High Quality Data**
70K conversations
(~800MB)




Helped launch academic open-source **GenAI research**

# LoRA Fine-Tuning

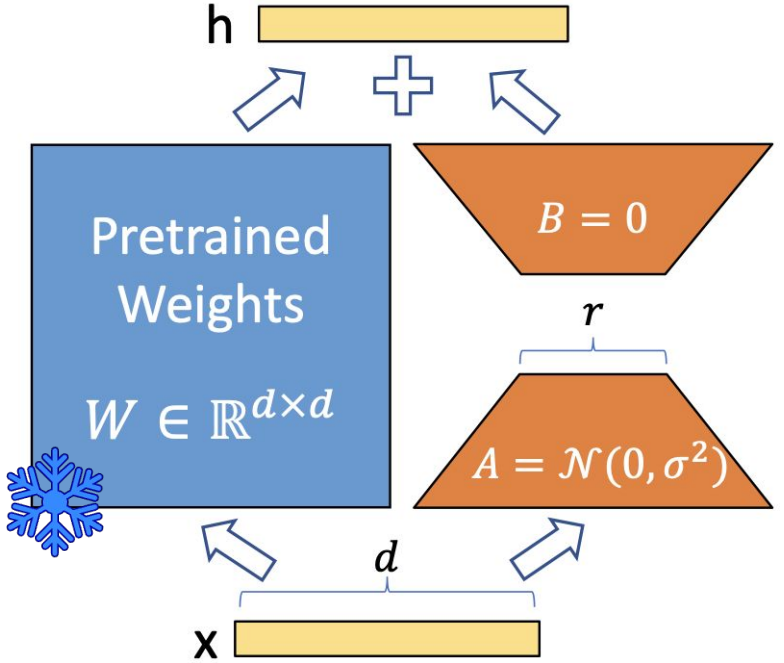
9688650

**Reduce costs and catastrophic forgetting by fine-tuning a low-rank perturbation:**

$$\mathbf{W}' = \mathbf{W} + \mathbf{A}\,\mathbf{B}\,x$$

where $\mathbf{W} \in \mathbb{R}^{D \times D}$ is the pre-trained weights and $\mathbf{A} \in \mathbb{R}^{D \times r}$ and $\mathbf{B} \in \mathbb{R}^{r \times D}$ are learned.

- The **B** weights are initialized to zero resulting in equivalent model (loss) prior to additional training.




Can be applied to **all weights** in the model.

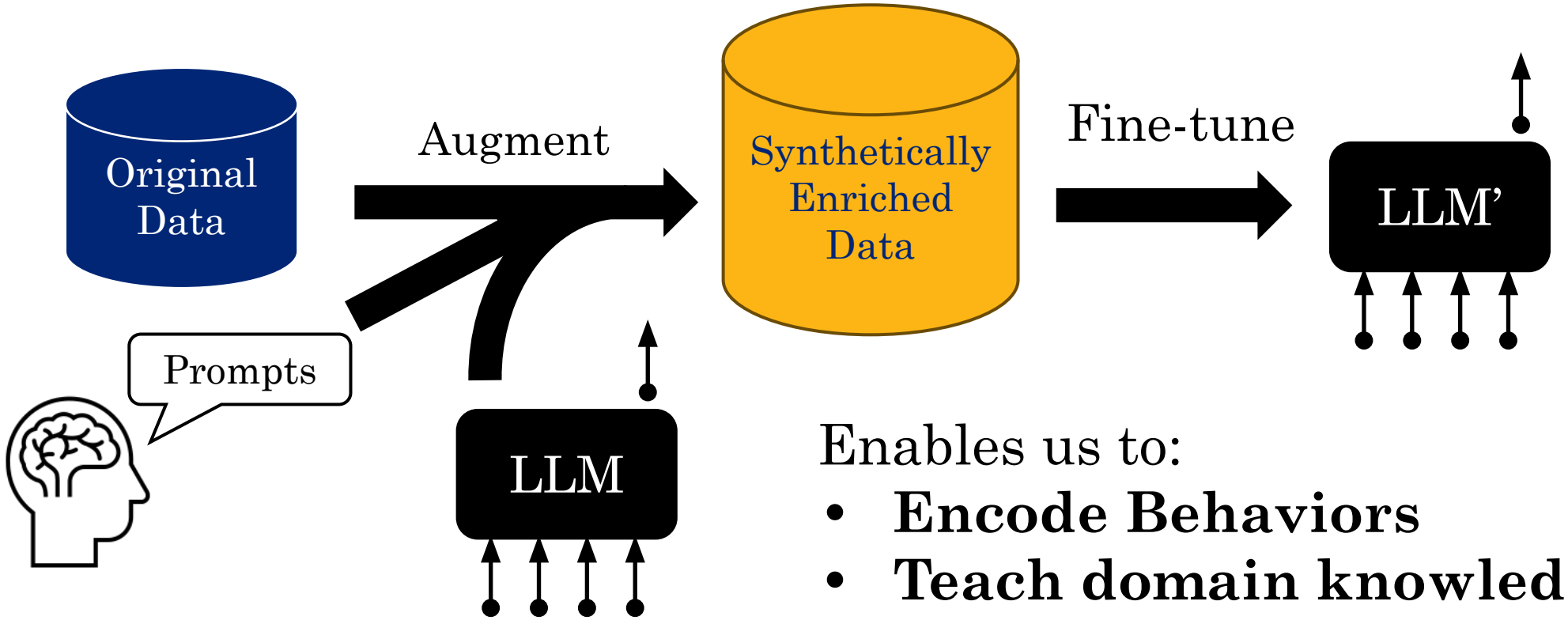
Reduces **training costs** and allows shared inference.

unsloth

# Synthetic Data Augmentation
## (Programming GenAI with GenAI)

9688650

Using an LLM to **augment data** to **fine-tune** an LLM



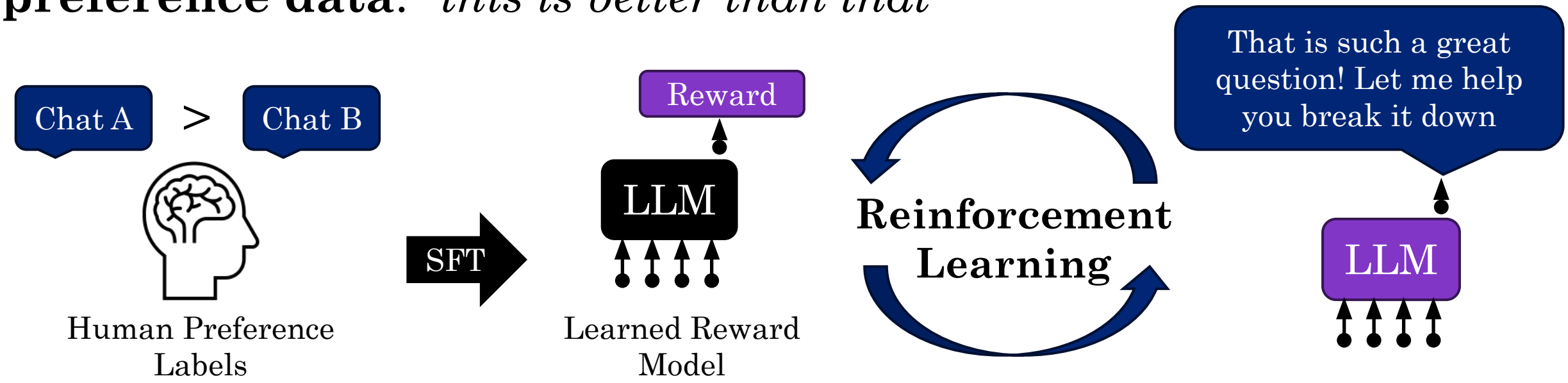

Enables us to:

- **Encode Behaviors**
- **Teach domain knowledge**

# Reinforcement Learning from Human Feedback (RLHF)

9688650

Can further **align model with human preferences** using **human preference data**: *"this is better than that"*

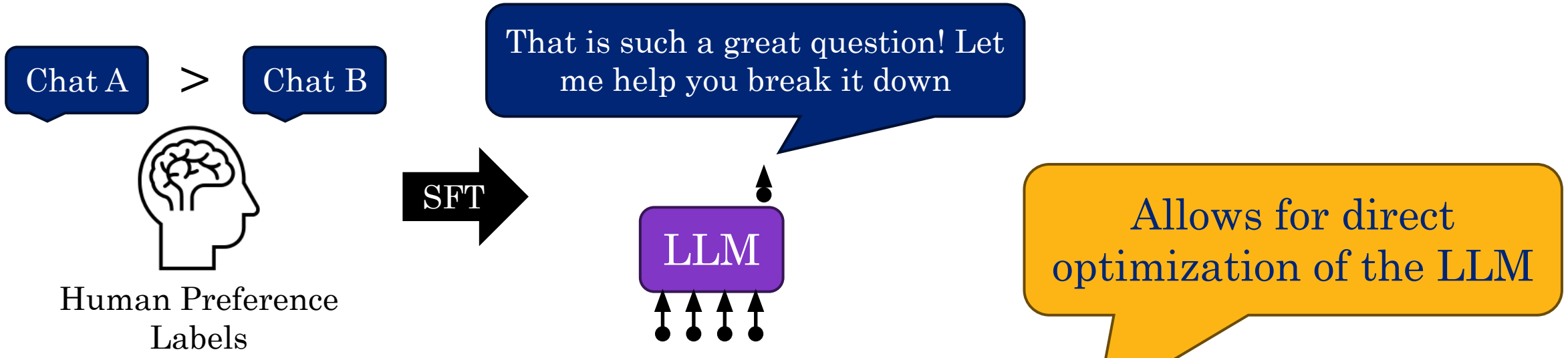



Helps to **make models more robust** and perform better in **safety situations** but can be **unstable** and **difficult** to train.

# Direct Preference Optimization (DPO)

9688650

Directly apply **Bradley-Terry Model** (HW2) to recast preference learning as **Maximum Likelihood Estimation**.




From the paper:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l)\sim\mathcal{D}}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right]$$

Don't need to memorize (just FYI).

https://arxiv.org/abs/2305.18290

# Prompting and Test-time Compute

Using context to guide models and extract capabilities through chain-of-thought

### Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  cheese =>                           ← prompt
```

Zero-shot relies on model already "knowing" how to complete the task.

### One-shot

In addition to the task description, the model sees a single example of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  sea otter => loutre de mer          ← example
3  cheese =>                           ← prompt
```

### Few-shot

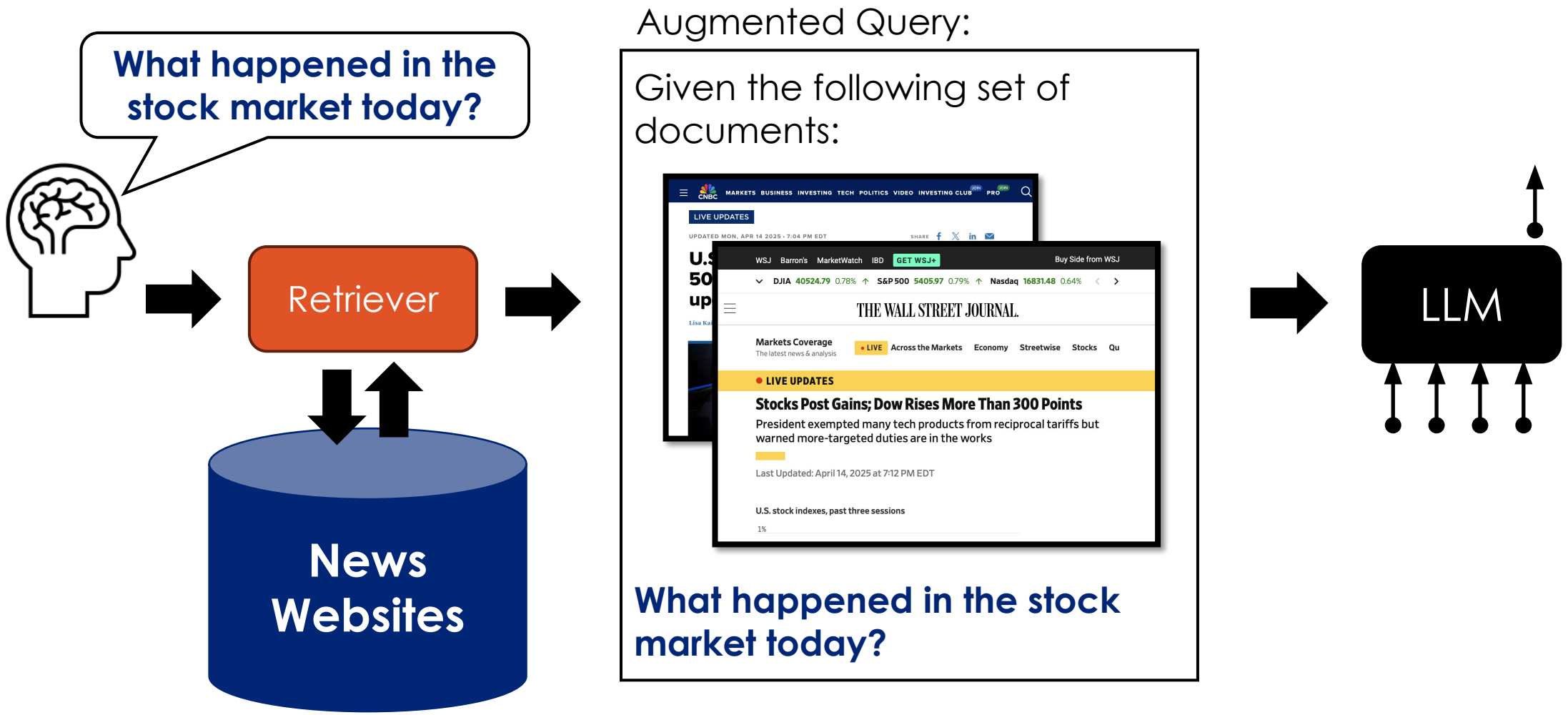
In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
1  Translate English to French:        ← task description
2  sea otter => loutre de mer          ← examples
3  peppermint => menthe poivrée        ←
4  plush girafe => girafe peluche      ←
5  cheese =>                           ← prompt
```

# In-context Learning

# Retrieval Augmented Generation (RAG)
## Giving LLMs Specialized Knowledge

9688650



# Chain-of-thought Prompting

9688650

**Standard Prompting**

**Input**

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

**Model Output**

A: The answer is 27. ✗

**Chain of Thought Prompting**

**Input**

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. 5 + 6 = 11. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

**Model Output**

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had 23 - 20 = 3. They bought 6 more apples, so they have 3 + 6 = 9. The answer is 9. ✓

# Reasoning in Large Language Models

9688650

Popularized by ChatGPT-O1

**Behavior:** leveraging long chain-of-thought with planning, reflection, self-refinement, and back-tracking.

**Prompt:**

```
How many of the numbers from the set $\{1,\ 2,\ 3,\ldots,\ 50\}$ have a
perfect square factor other than one?
```

**<|begin_of_thought|>**
So I have this problem: "How many of the numbers from …
**First**, I need to understand what a perfect square is. A perfect square is …
**One way to approach this** is to count the numbers that are divisible by each of these perfect squares …
Let's list out the perfect squares less than or equal to 50: …
**Next**, I'll find the numbers in the set \(\{1, 2, 3, \ldots, 50\}\) that are divisible by each …
Starting with 4:

**Prompt:**

`How many of the numbers from the set $\{1,\ 2,\ 3,\ldots,\ 50\}$ have a perfect square factor other than one?`

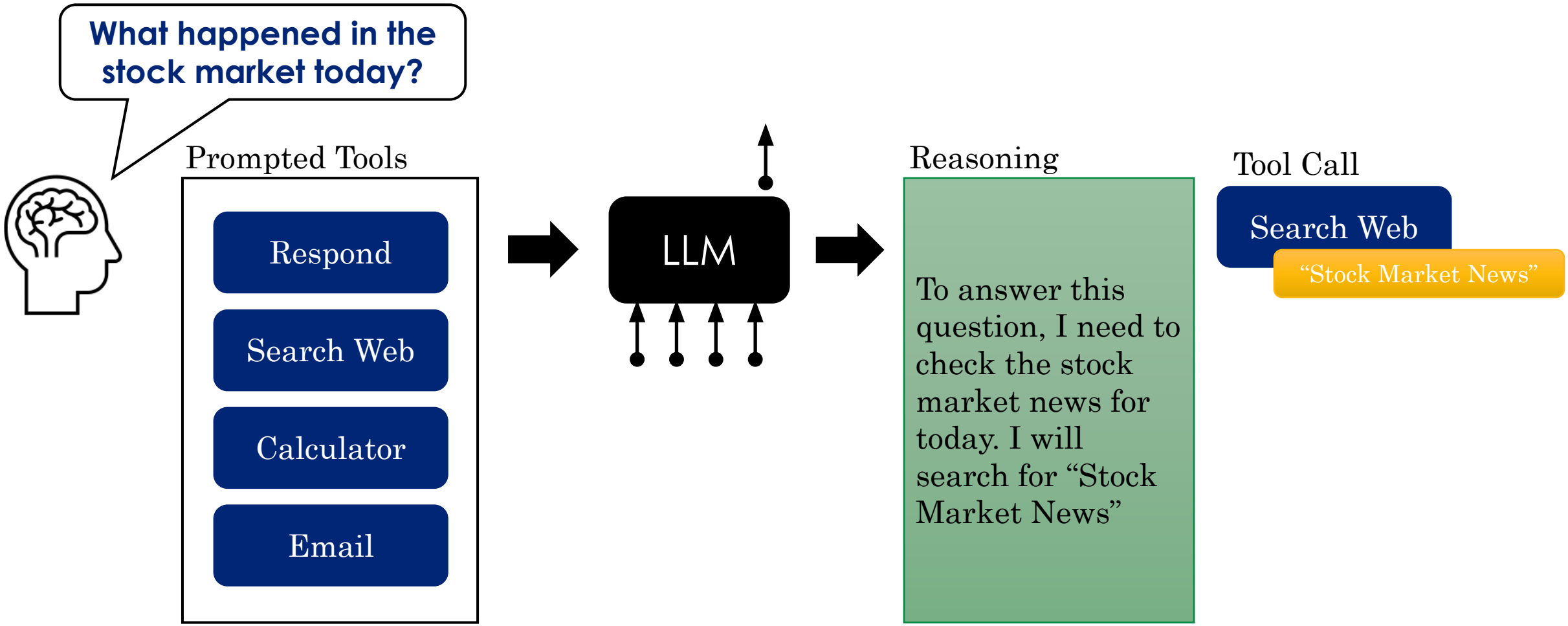
9688650

**<|begin_of_thought|>**
So I have this problem: "How many of the numbers from …
**First**, I need to understand what a perfect square is. A perfect square is …
**One way to approach this** is to count the numbers that are divisible by each of these perfect squares …
Let's list out the perfect squares less than or equal to 50: …
**Next**, I'll find the numbers in the set \(\{1, 2, 3, \ldots, 50\}\) that are divisible by each …
Starting with 4: …
…
If I add all these up: 12 + 5 + 3 + 2 + 1 + 1 = 24.
**But wait**, some numbers might be counted multiple times because they are divisible by more …
So, I need to use the principle of inclusion-exclusion to avoid double-counting. …
…
**But wait, let's double-check this** ….
…
**Alternatively, maybe I should** …
…
**Wait**, that's 20 numbers. But earlier I had 23. There must be a mistake in my earlier reasoning.
…
**So, the final answer is 19.<|end_of_thought|> <|begin_of_solution|> …**

450 lines (3000 words) of **"Reasoning"**

# Agentic Systems
## Enabling LLMs to ***explore the world***

9688650



# Agentic Systems
## Enabling LLMs to ***explore the world***

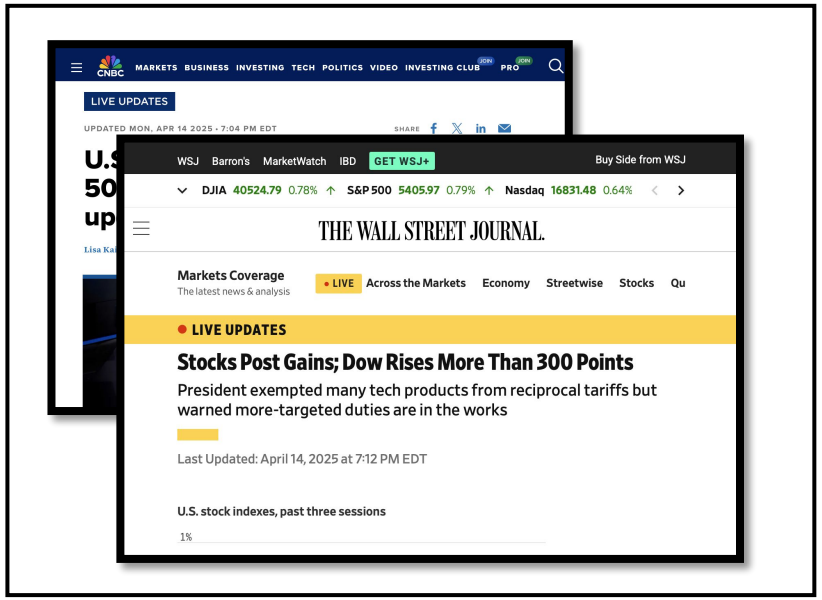
9688650

Tool Call

Search Web

"Stock Market News"

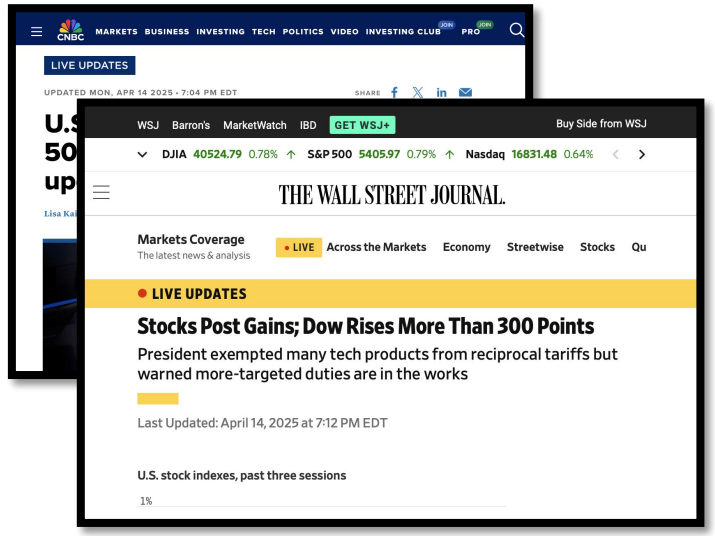
Tool Output



# Agentic Systems
## Enabling LLMs to ***explore the world***

9688650

Tool Output



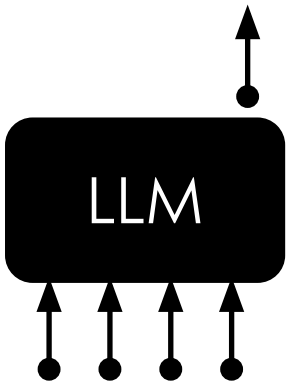

Histor

**What happened in the stock market today?**

To answer this question, I need to check the stock market news for today. I will search for "Stock Market News"

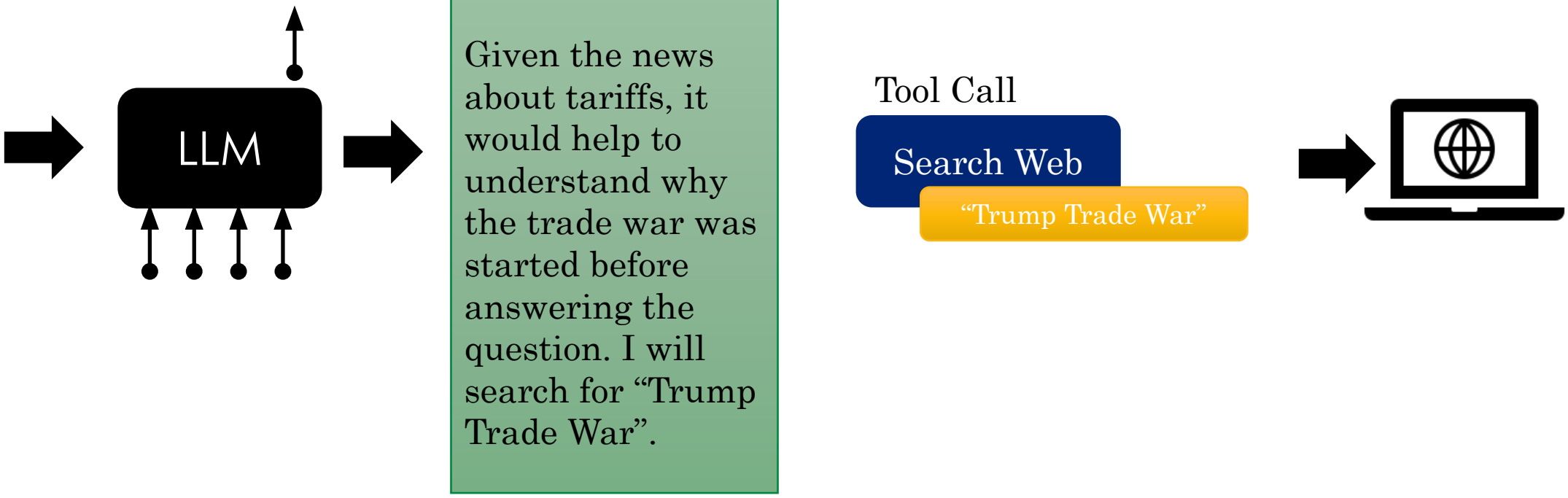
Prompted Tools

- Respond
- Search Web
- Calculator
- Email

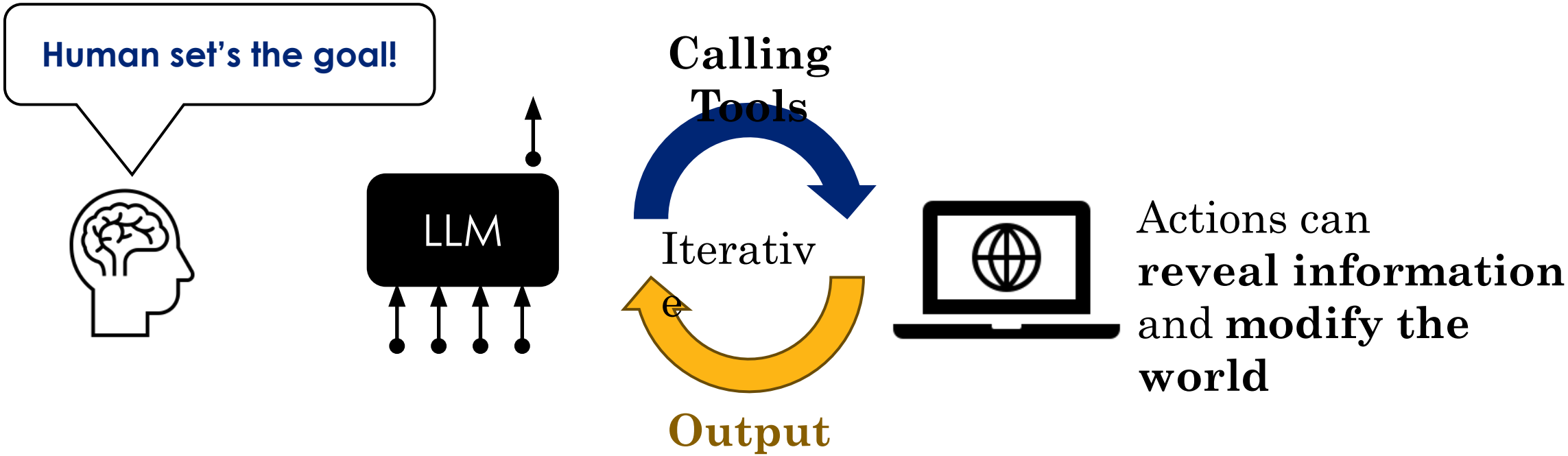


# Agentic Systems
## Enabling LLMs to ***explore the world***

9688650



# Agentic Systems
## Enabling LLMs to ***explore the world***




**Reason-Act (ReAct) Agents:** A lot of excitement about what these new systems can do.

Lecture 22

# LLM Training and Applications

**Reading:** *Chapter 10* in Bishop Deep Learning Textbook